

Master Thesis

Exploring LLM-Driven Enhancement Techniques for Machine Learning Pipelines Aimed at Improving Prediction Performance

Daniel van der Velde

Report submitted in partial fulfillment
of the requirements for the degree of
Master of Science of Data Science for Decision Making
at the Department of Advanced Computing Sciences
of the Maastricht University

Thesis Committee:

M.C. Popa
Dr E. Hortal Quesada

Maastricht University
Faculty of Science and Engineering
Department of Advanced Computing Sciences

June 29, 2025

1 Abstract

Recent breakthroughs in transformer-based architectures, most notably the Large Language Models (LLMs), have significantly reshaped the digital landscape in ways inconceivable in the recent past. While their success in natural language tasks is well-established, their role in enhancing traditional machine learning workflows remains underexplored. This work investigates how LLMs can support and improve ML pipelines, specifically transformer-based architectures. Key ML workflow tasks are attempted to be solved by LLMs directly, including, though not limited to, feature engineering, data synthesis, data augmentation, and feature selection. A modular framework is introduced to formalize seven hypothesized benefits LLM-driven enhancements may deliver.

This framework is empirically tested across three primary data modalities: tabular (UCI Adult Census), textual (IMDb Movie Reviews), and visual (SoCal House Prices and Images), using a diverse set of models including classical ML algorithms and neural networks. Moreover, an outline is given on how this framework may apply in differing data modalities and domains. Experiments are designed to compare baseline models and approaches with LLM-enhanced variants, assessing performance through standard metrics, predominantly by F1 and MAE. Findings illustrate that even small to mid-sized LLMs can yield tangible performance improvements in these contexts,

This work contributes to a growing library of LLM research by proposing a structured approach to integrating LLMs into ML workflows. It offers actionable insights and considerations for practitioners leveraging LLMs for their projects. It concludes by identifying conditions under which the LLMs could provide applied benefits, but it also outlines open challenges and limitations for actualizing future research into the topic.

2 Introduction

The past few years have reinvigorated the world of machine learning and artificial intelligence research. What was previously a gradual progression of minor incremental improvements suddenly became a paradigm shift. A central factor driving this transformation is a seemingly simple yet powerful architecture, known as the transformer, whose most famous offspring is the now widely adopted Large Language Model, or LLM for short.

This development has been accompanied by significant public discourse. It has generated considerable academic and societal debate, ranging from

copyright disputes and AI’s carbon footprint to fears of widespread job displacement and systemic economic disruption. While some concerns may be perceived as overstated, others are grounded in growing real-world implications. As technology advances, these debates will only become more apparent.

Notably, LLMs have already yielded some remarkable results. Tools like GitHub Copilot, powered directly by LLMs, have measurably increased productivity for knowledge workers [2]. Other transformer-based breakthroughs, such as AlphaFold 3 [1], have revolutionized protein structure prediction, demonstrating what this architecture might yield. Despite these successes, it remains evident that, for many industries, the impact of transformers is more about potential than practice.

This paper attempts to actualize that potential, particularly in traditional Machine Learning (ML). Is this field undergoing a substantial structural transformation due to this new paradigm shift? Or do LLMs and other transformer-based architectures still struggle to be of practical use in the ML workflow? More specifically, this paper aims to investigate how LLMs, and more broadly, the transformer architecture can support or enhance core ML tasks, such as feature and data generation, and help augment or improve the existing data. Can they strengthen model performance, and under what specific conditions? Additionally, what are the costs and trade-offs presented, and which data modalities experience the most gain in performance? Ultimately, this work aims to comprehensively map out the seemingly emerging role of LLMs throughout the ML pipeline, highlighting their practical utility and distinguishing it from prevailing hype.

Before exploring these questions in greater depth and formulating them into research questions, it is instructive to consider the broader context to understand why they matter in the first place. Previously confined to academic research, ML has increasingly integrated into various aspects of modern life, influencing consumer behavior, content exposure, and governance mechanisms. This phenomenon is sometimes referred to as the “Algorithmification” of life. With the advent of LLMs and transformer-based architectures entering the scene, the underlying mechanisms driving this transformation may be on the verge of significant disruption. If LLMs can enhance ML workflows and improve performance through this paper’s mechanisms, the downstream effects could significantly influence decision-making, operations, and behaviors through more efficient and capable algorithms.

The research questions posed in this paper are

formally presented as follows:

1. How can LLMs and transformer-based architecture enhance the performance of machine learning?
 - Can LLM-driven feature engineering improve model performance?
 - Can synthetic data generated by an LLM aid in transfer learning and generalize well to other datasets?
 - Can LLM-based feature selection outperform traditional feature selection techniques?
 - Can data augmentation using LLMs improve performance?
 - Could LLMs outperform traditional ML models?
 - What novel strategies can LLMs introduce to enhance model performance?
2. How should LLMs and transformers be cohesively integrated into data-driven decision-making processes and machine learning?

The scope of this paper will be mainly limited to the data modalities of tabular, textual, and image data. However, other data types will be considered, though no experimentation will be conducted on these. Additionally, this paper will not examine the inner workings of LLMs; the architectural design, pre-training, tokenization, and fine-tuning of the models are outside the scope of this research. However, it is worth considering the implications and opportunities presented by these techniques when evaluating the proposed data augmentations, domains, or modalities. To avoid relying on a configuration of an LLM that is too narrow, the paper includes a variety of LLMs to at least partially account for these variables. Furthermore, the paper only uses small to medium-sized LLMs to present realistic opportunities for ML practitioners. Ideally, the work would be replicated using larger LLMs to assess whether the results would differ. Lastly, the paper includes only a limited set of ML problems and cannot account for the wide variety of domains in which ML is applied. Therefore, the findings cannot be extrapolated to all domains. However, the paper does attempt to address which domains and ML problems could benefit the most from the allegedly attainable improvements.

This work contributes to the growing literature on the practical usage of LLMs by systematically and comprehensively investigating their integration into traditional ML workflows. It compares how LLMs can enhance model performance, particularly through feature engineering, data synthesis, and

data augmentation, across varying modalities, including tabular, textual, and image data. The paper highlights actionable considerations ML practitioners may apply when incorporating transformer-based architecture into their ML pipeline. The findings aim to inform researchers and practitioners on where, when, and how LLMs can add tangible value to ML and data-driven decision making.

3 Literature Review

In recent years, machine learning has undergone a significant shift, not just in terms of technical innovation, but in mentality. What previously was a field dominated by incremental progress has been reignited by a surge of excitement, mainly driven by the rapid advancement of LLMs and transformer architecture. This newfound momentum has increased funding and research, particularly from tech frontiers eager to benefit from AI opportunities. An illustrative example is the unprecedented investment in AI-specific data centers, which are now being constructed at large scales [11].

This increase in funding and research activity has significantly expanded the scope of AI research, with LLMs sitting at the center of it. Research output is exponentially increasing [6], and its focus has shifted from feasibility to applicability. This work examines one pivotal transformation angle of this transformation: incorporating LLMs into traditional machine learning workflows.

Several recent milestones illustrate the direction this is heading. Codex [5] and its successors serve as a particularly instructive example. These models address coding problems using natural language, reducing the distinction between human instructions and machine-executable solutions. Another noteworthy development is AutoML-Zero [13], which explores the use of evolutionary strategies and transformer architectures to develop machine learning pipelines from scratch. While these examples are promising, they prioritize automation over augmentation, aiming to replace parts of the workflow rather than enhance them. The potential for LLMs to support the model development process meaningfully is often overlooked: evaluating data quality, transforming data, engineering features, and tackling other sub-problems in the ML pipeline, possibly matching or surpassing current state-of-the-art solutions.

Some early attempts have been made to integrate LLMs into the ML pipeline, though with mixed results. A typical example is Smith et al. [16], which explores using LLMs to augment informal medical data for downstream deep learning tasks. While the authors concluded that LLMs

can be a powerful augmentation strategy, they cautioned that “the results will depend heavily on the implementation strategy used.” This finding is unsurprising in the context of generative AI: prompt engineering is a delicate task that demands a high degree of linguistic precision.

This challenge recurs throughout much of the current literature and highlights another key difficulty: domain adaptation. LLMs often struggle with domain-specific nuance when deployed in specialized fields like medicine. A broader survey by Gu et al. [7] echoes this concern, while also again noting the potential upside: “LLMs can indeed engineer meaningful features, automate data cleaning, synthesize new training samples, and reduce dependence on domain experts.” The study emphasizes the potential of LLMs to support various stages of the ML pipeline, provided that the right frameworks are in place. It also stresses the importance of modular systems accommodating LLMs cleanly and effectively. This study addresses this challenge directly.

Numerous studies have investigated improving various aspects of ML pipelines through LLMs or transformer-based architecture [8, 15, 4, 9, 20]. The papers investigate different parts of the ML workflow, from feature selection, hyperparameter tuning, and data generation. And while each provides unique insights, the recurring challenges remain pertinent: high computational costs, integration complexity, lack of standardization, and a growing need for domain-specific tuning. This paper focuses specifically on the need for integration strategies and standardization. The goal is to propose a general framework that can be iterated upon, highlighting where LLMs are most helpful, under what conditions, and for which types of tasks.

In summary, this work attempts to answer three simple but essential questions: Where do LLMs help? How much do they help? Which ML tasks are they best suited to? The aim is to draw general conclusions from a wide set of experiments, while perpetually noting the cost-benefit trade-off for real-world applications.

4 Methodology

This section introduces the proposed framework for enhancing key stages of the ML pipeline using LLMs and other transformer-based architectures. It is presented as a generalizable approach to integrating LLMs into ML workflows, guided by the central questions: how, when, and where can LLMs provide tangible improvements?

To explore and validate this integration, the framework will be applied to three distinct data

modalities: tabular, textual, and image data. The implementation details and modality-specific applications are elaborated in Section 7, focusing on identifying which LLM-driven benefits are relevant and how they can be actualized. Section 5 (subsection Other Data Types and the Template) outlines how the framework could extend to other data types, though these extensions are not empirically tested within this paper.

While Section 7 focuses on the technical execution and experimental design, the current section provides a conceptual foundation, emphasizing the rationale behind the framework rather than its mechanics. This high-level perspective sets the stage for the more detailed empirical investigations that follow.

Ultimately, these experiments will evaluate the framework to assess its effectiveness, identify limitations, and highlight opportunities for refinement. The resulting insights are intended to inform and guide future ML practitioners exploring the integration of LLMs into their workflows.

4.1 The Framework

This work proposes a modular framework designed to map, apply, and assess a range of LLM-based augmentation strategies across multiple data modalities to systematically evaluate the integration of LLMs into traditional machine learning workflows. The framework is structured around a collection of hypothesized benefits that LLMs may offer throughout the ML pipeline. Each benefit is conceptualized as an optional, insertable module. These modules can be applied independently or in combination, depending on the nature of the presented machine learning problem.

The framework identifies seven primary mechanisms, presented in no particular order, through which LLMs may enhance ML pipelines.

Important remark: The categories presented here should not be considered rigid or mutually exclusive. Their purpose is not to provide a comprehensive list but to serve as conceptual scaffolding, a reasoning tool to help ML practitioners identify where LLMs might add value. In practice, applications may span multiple categories, and some use cases of LLMs are not encompassed within this framework. It is designed to be modular and extensible, rather than exhaustive or prescriptive.

1. **Contextual/Semantic Enrichment:** LLMs can enhance datasets by injecting domain knowledge and world understanding. They can achieve this by generating novel features, inferring non-trivial insights from existing data, or combining different data

modalities through transformations.

2. **Disambiguation:** Data ambiguity is often a cause for degraded model performance. LLMs can reinterpret or rewrite ambiguous inputs (e.g., vague review texts) more discriminatively or instructively.
3. **Compression/Dimensionality Reduction:** LLMs can reduce data storage and RAM requirements through text summarization, replacing images with detailed image descriptions, or feature merging.
4. **Data Synthesis:** LLMs and diffusion models can produce synthetic samples in data-scarce or transfer learning environments. Validation protocols and LLM temperature could aid the integrity and diversity of such synthetic data.
5. **Robustness:** LLMs can correct noisy data, such as linguistic mistakes, inconsistent tabular fields, or visual artifacts in images, making it easier for downstream models to focus on relevant patterns and reducing variance in performance. While many traditional methods exist to improve data robustness, LLMs can offer a distinct advantage in addressing non-trivial robustness challenges.
6. **Feature Selection:** LLMs can mimic domain experts by selecting the most predictive features.
7. **LLM as Model:** LLMs can also be used directly to classify or predict outcomes via zero-shot or few-shot prompting, bypassing the need for data engineering and modelling.

While this list captures the primary performance-related enhancements explored in this work, several additional benefits LLMs may provide are outside the scope of this work, but some are nonetheless worth noting.

1. **Fairness:** LLMs could assist in identifying or mitigating bias in datasets or model predictions, for example, by detecting hate speech or flagging skewed variable importances. More advanced techniques, such as LLM-guided fairness-aware model explanation, are also possible [14].
2. **Explainability/Interpretability:** LLMs can be prompted to produce human-readable explanations of model predictions. For example, by providing both a data sample and its associated feature importances, one could instruct the LLM to reason through how the

model may have arrived at its conclusion, thus reducing the opacity of the ML pipeline.

Finally, while this work focuses exclusively on off-the-shelf LLMs, it is worth noting that LLMs can also be fine-tuned. Fine-tuning could enhance performance across the enhancement techniques from the template, especially in domain-specific contexts. However, this direction is excluded from the current scope and is suggested as a potential avenue for future work.

5 Data

This paper evaluates the impact of LLM-assisted workflows across various data modalities: tabular, textual, and visual. It expanded on three publicly available benchmark datasets, each selected to represent a typical use case in modern machine learning research. The chosen datasets are widely used and well-documented, ensuring reproducibility and relevance. Below, we detail their structure, motivation, and role in the experiments.

Finally, there will be a subsection considering the application of the template outlined in Section 4 to other common data types.

5.1 Tabular - UCI Adult Census Income Dataset

The Adult Census Income dataset, commonly called the “Adult” or “Census Income” dataset, is a resource for studying classification algorithms and algorithmic fairness. Curated initially from the 1994 U.S. Census database, the UCI Machine Learning Repository hosts it [3] and contains 48,842 rows. Each entry corresponds to an individual and is described by 14 features that capture key demographic and economic attributes. These include variables such as age, sex, marital status, race, education level, occupation, and weekly working hours. The predictive task centers on a binary target variable indicating whether the individual’s annual income exceeds \$50,000.

This dataset is particularly valuable because of the ethical and social dimensions embedded in the data. It has become a benchmark for evaluating classification performance, interpretability, and fairness. The presence of sensitive attributes like gender and race has led to widespread adoption in bias mitigation and fairness-aware modelling studies.

For this paper, the Adult dataset is a representative case of structured, tabular data in a real-world setting. Its categorical and continuous feature mix makes it well-suited for exploring how

LLMs can support the traditional machine learning process. Furthermore, because the dataset has been extensively studied, its inclusion ensures that results from LLM-enhanced workflows can be compared against established baselines and prior work with clarity and credibility.

5.2 Textual - IMDb Movie Reviews

To evaluate LLM-assisted workflows in unstructured textual data, this work utilizes the IMDb Movie Reviews dataset [12]; a commonly used benchmark in sentiment analysis. The dataset comprises 50,000 English-language reviews sourced from IMDb, with an even split between positive and negative sentiment labels. Each review reflects a user’s opinion about a film, offering diverse linguistic structures, varying lengths, and rich subjective content.

The dataset’s preprocessed nature, where reviews are stripped of extraneous metadata, non-textual features, and noise, enables focus on core modelling tasks. It avoids many practical challenges of web-scraped text, such as HTML parsing or tokenization of emojis. Instead, it provides a controlled environment to study how models engage with semantic content. Its class balance makes it especially appropriate for evaluating classification performance without the confounding effects of skewed distributions.

In the context of this paper, the IMDb dataset serves as a test bed for examining how LLMs can enhance or transform traditional Natural Language Processing (NLP) workflows.

5.3 Image - SoCal House Prices and Images

This paper employs the “House Prices and Images - SoCal” dataset from Kaggle [17] to explore how LLMs can augment machine learning workflows involving visual data. This dataset poses a realistic and multifaceted problem: predicting residential property prices using images and structured metadata. Each sample consists of an image of a Southern California house, alongside its market price. In addition to the visual data, each listing includes tabular attributes such as the number of bedrooms and bathrooms, square footage of the house, and the city it is located in. Images are standardized at 415×311 pixels with three RGB channels.

Unlike traditional vision datasets designed for classification tasks (e.g., object detection or scene recognition), this dataset presents a regression challenge: predicting a home’s monetary value based on visual input. The task’s complexity arises from the

need to infer pricing cues from features such as architectural style, curb appeal, material quality, and even neighbourhood context visible in images.

This dataset is particularly well-suited for evaluating the role of vision-large-language models (VLLMs), and this paper specifically focuses on those from the Ollama arsenal. While traditional LLMs operate on text, VLLMs can process images directly as encoders for downstream tasks or as interactive agents capable of generating explanations, ranking listings, or identifying visual patterns relevant to pricing. These models enable novel multimodal pipelines in which image, tabular, and textual inputs are integrated for prediction, interpretation, and feature synthesis.

Importantly, this work does not treat the dataset as unilateral. The experimental design explores multiple configurations, including image-only, multimodal (image + tabular), and even extends to efforts at reintegrating textual data as an additional modality, as detailed in Section 7. This setup enables an investigation of how VLLMs perform across different data modalities, assessing their ability to enhance machine learning pipelines through the proposed benefits.

5.4 Other Data Types and the Template

This subsection explores the theoretical applicability of the framework outlined in Section 4 to other commonly encountered data types. While the scope of this paper is limited to empirical evaluation on three primary data types, it is nonetheless valuable to consider how the framework might generalize. In particular, similarities between data types can be used to argue for the framework’s potential usefulness in future research. Below, we briefly examine how the template might extend to additional data types:

- **Video data:** Video data can be interpreted as a sequence of images. Similar enhancements may be feasible for video data if the proposed enhancements yield benefits for image data. More concretely, the framework could be applied on a per-frame basis. However, a significant methodological challenge would be the numerous VLLM interactions required for the many video frames. This could be mitigated by applying the enhancement strategies at a fixed interval, e.g., every X frames, rather than continuously.
- **Graph data:** Graph data consists of vertices and edges. The vertices often have tabular attributes; for example, in a social network, each node might represent a person

with features such as age, height, and ethnicity. These tabular features are suitable candidates for enhancement using the proposed template. Whether the edges (i.e., the connectivity information) can similarly benefit from LLM-based enhancements remains an open research avenue.

- **Time series data:** Time series data often manifests in tabular formats (e.g., weather data) or as images (e.g., plant growth over time). In such cases, the proposed enhancement strategies could be readily applied. However, fully leveraging the temporal dynamics of time series data would require an extension of the current template to account for temporal dependencies.
- **Audio data:** Audio can be approached in two ways: as time series data, in which case the previous point applies, or as textual data through transcription. In the latter scenario, the framework could be applied directly to the transcribed text, making it a viable candidate for LLM-based enhancement.
- **Geospatial and 3D data:** This category encompasses a wide variety of formats, each with distinct representational options:
 - Geospatial raster data (e.g., satellite imagery) can be treated as image data and treated by the template accordingly.
 - Abstract geospatial structures (e.g., road or river networks) can be represented as graphs, making them compatible with the graph-based application of the framework, as argued in this subsection.
 - Spatio-temporal geospatial data (e.g., evolving weather patterns) resemble video data and may benefit from the same interval-based (image) enhancement strategy.
 - 3D data (e.g., MRI scans, terrain density maps) present the most significant challenge. One potential approach is to project or flatten 3D data into 2D representations, effectively transforming them into images. While this simplification may allow partial template application, it will likely be lossy and context-dependent.

In summary, while the template’s current implementation is limited to specific data types, its conceptual foundation appears adaptable. With further development, particularly around temporal

and 3-dimensional data, it may serve as a generalizable strategy across a broader spectrum of data modalities.

6 Models

This section outlines the models selected for the experiments and justifies their inclusion. Traditional ML models were primarily applied to classification tasks involving textual and tabular datasets, whereas neural networks were used for the regression task, predicting house values across various data modalities. Additionally, LLMs, VLLMs, Stable Diffusion (SD) models, and a Generative Adversarial Network (GAN) were incorporated to enhance different stages of the ML pipeline, in line with the hypothesized framework presented in Section 4. Tables 1 and 2 summarize the selected machine learning models and neural network architectures, respectively. Tables 3 and 4 detail the LLMs, VLLMs, SD models, and GAN employed for enhancement tasks. For reference, the specific architectures of the neural networks listed in Table 2 are provided in the Appendix (Section 13).

Traditional ML Models	
Abbreviation	Model Name
LR	Logistic Regression
RF	Random Forest
SVM	Support Vector Machine
KNN	K-Nearest Neighbors
GB	Gradient Boosting

Table 1: Traditional machine learning models tested. The default parameters used with seed 42.

LLMs for Tabular/Textual Data	
Model	Parameters
LLaMA3.2	1B, 3B
LLaMA3.1	7B
Gemma3	1B, 4B
Mistral	7B
Dolphin3	8B
Deepseek	7B

Table 3: Language models used for tabular and textual data processing with parameter counts.

Neural Networks				
Model	Data types	Total Params	Trainable Params	Figure
NN Base	Image	109,889	109,889	Figure 28
NN Base	Image, Tabular	122,529 (122,849 for FG)	122,529 (122,849 for FG)	Figure 29
NN Base	Image, Tabular, Text	2,703,329	2,703,329	Figure 30
ResNet	Image	23,850,113	4,728,065	Figure 31
ResNet	Image, Tabular	23,862,753 (23,863,073 for FG)	4,740,705 (4,741,025 for FG)	Figure 32

Table 2: Used neural network architectures.

Vision LLMs	
Model	Parameters
LLaVA	7B
LLaVA-LLaMA3	8B
Gemma3	4B

Diffusion Models/GAN	
Model	Parameters
Stable Diffusion v1.4	860M
Stable Diffusion v1.5	890M
Stable Diffusion v2.1	1.0B
RealESRGAN	16M

Table 4: Vision models, diffusion models, and GAN used for image data processing with parameter counts.

6.1 Machine Learning Models

The following standard machine learning algorithms were selected: Logistic Regression, Random Forest, K-Nearest Neighbors, Gradient Boosting, and Support Vector Machines (Table 1). These models were applied to the classification tasks involving tabular and textual data. They were chosen due to their widespread use in academic and applied research and their well-documented performance across various tasks.

All models used default parameters from the scikit-learn Python library, with a fixed random seed 42 to ensure reproducibility. No hyperparameter optimization was performed, as the models are intended to serve as baselines. The aim is to provide neutral reference points for evaluating the hypothesized performance gains introduced by LLM-based enhancements.

6.2 Neural Networks

Two families of neural networks were developed and evaluated for the regression task of predicting house prices: a custom architecture referred to as NN Base, and a deeper, pretrained architecture based on ResNet50.

NN Base is a modular architecture implemented in three configurations:

1. **Image-only:** A convolutional neural network (CNN) that processes house images.
2. **Image + Tabular:** A hybrid model combining a CNN branch for image inputs with a fully connected branch for tabular data. The outputs of both branches are concatenated before the final regression layers.
3. **Image + Tabular + Text:** A multimodal extension that adds a third fully connected branch for Bag-of-Words (BOW) textual input. Outputs from all three branches are concatenated before the final regression layers.

ResNet-based models use a pretrained 50-layer ResNet architecture and are implemented in two variants:

1. **ResNet (Image-only):** Regresses only image inputs through ResNet50.
2. **ResNet + Tabular:** A hybrid model in which ResNet handles image inputs, while a separate fully connected branch processes tabular features. The outputs are concatenated before the final regression layers.

In all ResNet variants, the final 10 layers are unfrozen to enable task-specific fine-tuning, allowing for reasonable training times while still adapting the model to the regression task. ResNet was selected for its ability to extract deep latent representations from visual data, capturing complex patterns that may be beyond the reach of the shallower NN Base models. Moreover, the ResNet50 was originally trained on a dataset that includes house images, suggesting it likely already encodes useful house-related features.

The NN Base and ResNet architectures were modified during the feature generation (FG) experiment, and five additional features were introduced to the tabular branch. This addition increased the parameter count by 320 (i.e., 5 features $\times$ 64 weights per feature).

6.3 LLMs, SD, and RealESRGAN

A range of LLMs was used in this paper, with model sizes spanning from approximately 1 billion to 8 billion parameters. This is considered small to lower mid-range for contemporary LLMs, and they can be run efficiently on modern consumer-grade hardware, particularly when equipped with a dedicated GPU and CUDA support [21]. On a typical mid-range GPU, inference times ranged from 1 to 15 seconds per output, depending on the input and output token counts.

A core objective of this paper was to investigate the minimal viable LLM size that still yields measurable performance improvements in machine learning workflows. A key motivation was to evaluate the practicality of deploying these models on non-specialized hardware, assessing whether the average machine learning practitioner can meaningfully benefit from LLM integration without access to expensive server infrastructure.

Different models were selected based on their suitability for specific modalities to accommodate the diverse data types used in the experiments. Table 3 lists the LLMs applied to textual and tabular data tasks, while Table 4 includes vision models used for image-based tasks. The latter also features diffusion models and a GAN model employed in supplementary image enhancement experiments. Unless otherwise stated, all models were run using their default settings (e.g., temperature values).

Model selection prioritized accessibility and relevance. All chosen LLMs are lightweight enough to be run locally on standard hardware, while still representing the current state of the art (for the parameter range of 1 to 8 billion) from major research labs, including Google and Meta. This same rationale guided the selection of VLLMs. The Stable Diffusion models were also selected for their widespread adoption and low computational requirements, which aligns with the hardware constraints set in this paper. Finally, RealESRGAN was chosen for its strong performance in real-world image super-resolution tasks [18]. Unlike traditional super-resolution approaches, RealESRGAN is specifically designed to handle complex and mixed image degradations, such as noise, compression artifacts, and blur, which were frequently observed in the SoCal image dataset.

All LLMs and VLLMs were accessed via the Ollama interface, which provided a standardized aggregation layer. Ollama abstracts the complexities of interacting with different models by offering a unified prompting interface and consistent usage behavior. This approach ensured reproducibility and minimized variability across the back-end. All experiments used Ollama version 0.6.4.

7 Experiments

Experiments were conducted on three distinct data types, tabular, textual, and image data, each aligned with a specific machine learning task.

- **Tabular data:** Binary classification to predict whether an individual earns more or less than \$50,000 annually, based on demographic and occupational characteristics.
- **Textual data:** Sentiment classification of movie reviews, identifying whether a review expresses a positive or negative sentiment.
- **Image data:** Regression to estimate property prices using exterior images of houses, augmented with relevant tabular characteristics (e.g., square footage, number of bedrooms).

The tasks serve as applied test beds to evaluate the proposed advantages of utilizing LLMs and transformer-based architectures across various data modalities. The theoretical foundations supporting each identified advantage are delineated in Section 4. This section presents a comprehensive account of the experimental configuration employed to validate these advantages empirically.

The experiments are organized around the categories of hypothesized LLM benefits, as outlined in Section 4. These categories are briefly revisited below to contextualize their application within the experimental setup:

- a) **Contextual/Semantic Enrichment:** Enhancing the original data by incorporating domain knowledge, additional facts, or inferred insights. This approach often involves making non-trivial connections beyond surface-level patterns, enriching the dataset with information it would otherwise lack. This extra information can potentially improve models trained on these enriched datasets.
- b) **Disambiguation:** Clarifying or restructuring data to reduce ambiguity makes it easier for models to learn and generalize from meaningful patterns. For instance, LLMs may be used to rewrite movie reviews to preserve their original sentiment while making the language more descriptive. This clarification can introduce more sentiment-relevant cues, such as emotionally charged words, that enhance model performance.
- c) **Compression:** Reducing the volume or complexity of the input data through summarization or even by changing the data modality.

A long article might be summarized into key points, or an image might be described in detailed text instead. Although this can lead to a minor reduction in accuracy, it typically results in faster models that require less computational and storage overhead.

- d) **Data Synthesis:** Generating novel data samples from scratch or through simple heuristics. LLMs can simulate additional examples to supplement limited datasets or produce synthetic edge cases that improve model robustness and generalization.
- e) **Robustness:** Refining low-quality data by correcting spelling mistakes, managing outliers, or enhancing noisy visual inputs. This cleaning process makes it easier for downstream models to focus on the relevant features and can lead to more stable, reliable learning outcomes.
- f) **Feature Selection:** Leveraging LLMs to identify and retain only the most predictive features. By removing redundant or noisy inputs, this process helps reduce multicollinearity and improve model interpretability and performance.

It is worth noting that these categories are not mutually exclusive. In practice, several of these strategies may overlap. For example, contextual enrichment, disambiguation, and robustification can all be viewed as variants of data augmentation, differing mainly in their underlying motivation or mechanism. The purpose of these categories is not to establish rigid boundaries, but to offer applicable mental models for model engineers to help reason through and identify effective data augmentation strategies.

Finally, not all benefits are equally applicable across all data modalities. For instance, tabular data typically offers limited scope for disambiguation. These nuances and each category’s limitations and practical considerations are discussed further in Section 4.

7.1 Tabular Data

All experiments on tabular data were conducted using the UCI Adult dataset [3], which has 48,842 samples, of which roughly 76% of the individuals earn less than \$50,000 annually (target variable). The dataset was partitioned into two subsets: the LLM-augmented and base datasets. The base and LLM-augmented datasets are maintained as equal in size as possible for a fair comparison. The LLM-augmented data set is equivalent to the base data

set, but also consists of samples modified by specific LLM-based augmentations, as detailed in Section 4.

The base and LLM-augmented datasets are subsets of the whole dataset, each split into training and testing sets using an 80/20 ratio and a fixed random seed to ensure consistent instance selection across both versions. Note that while these subsets were not stratified, this does not affect the validity of the comparison, as both datasets share the same partitioning and class distribution variability. Lastly, a baseline dataset (whole dataset) is always used to validate and affirm that the base dataset (subset) is representative.

Standard measures were used for assessing model performance:

1. Accuracy
2. F1-score
3. Precision
4. Recall

For the Data Synthesis (DS) experiment, two additional metrics were included to assess the quality of the generated data:

1. Average Pairwise Distance (APD)
2. Cumulative Explained Variance (CEV)

Further details about these metrics are provided in the subsequent subsection.

A modified set of LLMs was employed in each experiment. Usually, a default temperature of either 0.7 or 0.8 was adopted, consistent with the default settings of the Ollama models [10]. However, different temperature conditions were investigated for DS to maximize the APD and CEV values.

Lastly, a standardized set of machine learning models was used to test the proposed advantages of LLM-based augmentations. The complete set of models is displayed in Table 1, and an overview of the entire experimental setup for tabular data is provided in Table 5.

The following subsections briefly describe the purpose of each experiment, along with any additional clarifications or nuances specific to them. It is recommended to cross-reference these subsections with Table 5, which summarizes key parameters used in each experiment.

Tabular Data Flow

A flowchart is presented in Figure 1, which will be briefly discussed here to clarify the process behind the tabular experiments. The workflow begins with

Table 5: Experimental Configuration for Tabular Data.

Experiment	LLM-Augmented Dataset Size	Base Dataset Size	Performance Metrics	LLMs Used (see Table 3)	MLMs Used (see Table 1)
Data Synthesis	5,000 train, 9,769 test	100 train, 9,769 test	Accuracy, F1, Precision, Recall, APD, CEV	LLaMA3.1:7B, Mistral:7B (temps 5, 10, 15)	All
Contextual / Semantic Enrichment (FG)	4,000 train, 1,000 test	4,000 train, 1,000 test	Accuracy, F1, Precision, Recall	All	All
Feature Selection W/O FG	39,073 train, 9,769 test	39,073 train, 9,769 test	Accuracy, F1, Precision, Recall	All	GB
Feature Selection With FG	4,000 train, 1,000 test	4,000 train, 1,000 test	Accuracy, F1, Precision, Recall	All	GB
LLM as Classifier W/O FG	5,000 test	43,842 train, 5,000 test	Accuracy, F1, Precision, Recall	All	All
LLM as Classifier With FG	5,000 test	4,000 train, 1,000 test	Accuracy, F1, Precision, Recall	All	All

the full raw dataset, which undergoes a series of pre-processing steps: categorical variables are encoded, redundant columns are removed, missing values are filled using a dedicated categorical token, and numerical data is ratio-scaled. The result is the base data, the foundation for training classification models.

These models were then evaluated against counterparts trained on synthetic data, as outlined by the corresponding data synthesis experimental setup. In parallel, the base data was processed by an LLM-based feature generation process, producing additional features as described in the contextual/semantic enrichment experiment. This results in the LLM-augmented data, which was used to train separate models. Comparisons were made between models trained on the base data and those trained on the augmented data.

The base and augmented data were also transformed into natural language representations using NLG techniques in the LLM as a classifier experiment. The feature selection experiment also leveraged these exact textual representations, allowing the LLM to assess and prioritize features based on their semantic descriptions.

Data Synthesis

This experiment examines the efficacy of using LLM-generated training data to train downstream ML models in a controlled Transfer Learning setting. Three datasets are generated for each LLM, each corresponding to a different temperature setting and containing 5,000 valid synthetic training samples. Only training data samples are generated; the test samples are taken from the base data.

All LLMs in Table 3 were initially evaluated. Still, only two models demonstrated acceptable performance: LLaMA3.1:7B and Mistral:7B. Smaller models, particularly the 1B and 3B variants, pro-

duced unusable data in nearly all cases, with failure rates approaching 100%. Similarly, models such as Gemma3 and DeepSeek exhibited error rates exceeding 50%, primarily due to structural or semantic inconsistencies. Only LLaMA3.1:7B and Mistral:7B managed to generate usable data with $\geq 80\%$ validity, which is still high but low enough to work with.

These high error rates largely stem from each complete tabular sample being generated in a single request. When many columns are involved, the probability of introducing an error in at least one field increases substantially. To ensure dataset consistency, synthetic data was generated iteratively until exactly 5,000 valid samples were collected for each model-temperature pair. Sample validity was assessed using simple heuristics, for example, verifying that the “age” feature was returned within the range of 18 to 100, or that the “occupation” field matched one of the predefined categorical values. Additional implementation details are provided in Section 13.4.

Importantly, the baseline and LLM-augmented datasets were evaluated using the same non-synthetic test set. No synthetic data was used during testing, ensuring that performance comparisons reflect generalization to real-world data.

To assess the diversity and structure of the generated datasets, two additional metrics were employed:

1. **Average Pairwise Distance (APD):** The average distance of each sample (row) to the others in the dataset. Note that the samples are strictly numerical, as the data was pre-processed. Categorical values were encoded using one-hot or label encoding, and the dataset was normalized before calculating this score. Lower APD indicates greater similarity between samples, which may lead to decreased data diversity and cause models

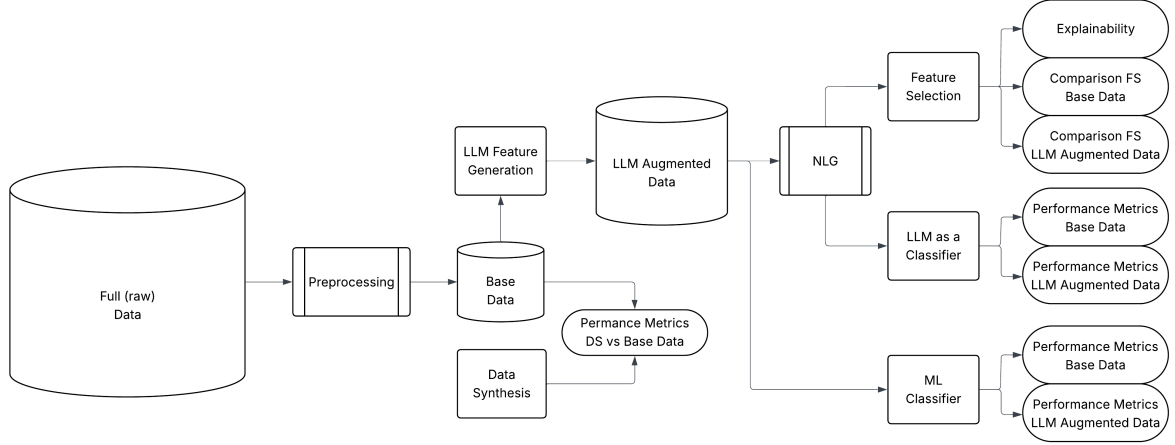


Figure 1: Tabular Data Flow per LLM (Table 3).

trained on this data to generalize poorly. Importantly, this metric should not be used as a proxy for model performance, as increased data diversity may not correlate with sample validity.

2. **Cumulative Explained Variance (CEV):** Calculated using Principal Component Analysis (PCA), this metric quantifies the proportion of total dataset variance explained by the first three principal components. A low CEV means it takes more features to explain the data’s variation, suggesting more complexity and diversity in the generated data. Again, this metric only indicates the variation and complexity of the dataset, which may not correlate with the validity of the generated data.

The results section will also report the observed error rates for each LLM-temperature configuration, providing insight into the trade-off between sample diversity and validity.

Contextual/Semantic Enrichment

The goal of this experiment is to explore whether the incorporation of additional contextual features in the LLM-augmented dataset improves the classification performance. To explore this further, the primary database was supplemented with an array of socio-economic and career-focused characteristics, such as:

- **Career Stage Classification:** Categorizes the individual as being in their professional career’s early, mid, or late stage.
- **Occupational Demand Outlook:** Estimates future labour market demand for the individual’s occupation.

- **Education ROI:** Assesses the return on investment of the individual’s educational background.
- **Years of Experience:** Approximated based on age and typical educational timelines.
- **Socioeconomic Mobility:** Evaluates the individual’s potential for upward economic mobility.
- **Job-Education Alignment:** Measures the degree of congruence between the individual’s job and educational qualifications.
- **Job Security Rating:** Indicates the individual’s current occupation’s perceived stability and long-term security.
- **Cultural Integration Score:** Estimates the individual’s potential to integrate successfully into their country’s sociocultural environment.

Each feature was derived from structured prompts (see Table 8), which asked to score each feature from 1 (low) to 10 (high) given the individual’s profile (a row in the tabular dataset). The scores obtained were standardized and used as input features for classical machine learning models. The enrichment performance is evaluated by comparing models trained on the enriched (augmented) dataset with those trained on the base dataset (without LLM-generated contextual features).

Every LLM computes its version of the dataset with the corresponding constructed features, which are evaluated separately. Additionally, this experiment investigates the effect of merging all LLM-generated features into a composite dataset for evaluation.

Feature Selection

This work evaluates the effectiveness of Large Language Model-based Feature Selection (LLM-FS) relative to several state-of-the-art (SOTA) feature selection (FS) techniques. When appropriately prompted, the underlying assumption is that LLMs can approximate a domain expert’s (DE) reasoning in discerning relevant from irrelevant features in a machine learning task context. If this hypothesis is valid, LLM-FS performance should coincide closely with established FS methods.

To test this, LLM-FS is compared against three widely used classical techniques: Shapley-based Feature Selection (Shapley-FS), Sequential Feature Selection (SFS), and Recursive Feature Elimination (RFE). All comparisons are conducted using Gradient Boosting (GB) models, selected based on superior performance observed in preliminary benchmarking experiments.

FS via LLMs is performed through standardized prompts (see Section 13.5) to extract the top 5, 10, and 15 most relevant features. In cases where the LLM fails to return the exact number of requested features, the subset of valid features is returned as-is. These instances are reported in the results under the label “failed models” (e.g., LLMs that fail to produce the exact amount of features by returning fewer features than requested) and are included to maintain the integrity of the evaluation process.

This experiment also incorporates findings from the earlier Semantic Enrichment experiment. Specifically, FS methods are applied to both the original dataset and a semantically enriched version that includes LLM-generated features. Accordingly, the analysis is divided into two sub-tasks: Feature Selection Without FG, using the baseline dataset, and Feature Selection With FG, using the enriched dataset. Due to the computational and design constraints discussed earlier, the enriched dataset is limited to 5,000 examples, making it notably smaller than the dataset used in the Without FG condition. This design enables us to evaluate the absolute and relative performance of LLM-FS in different feature space conditions, standard versus semantically enriched, and against established selection benchmarks.

LLM as Classifier

This experiment investigates how well an LLM can serve as a zero-shot classifier when presented with natural language representations of structured tabular data. To facilitate this, each row of the original dataset was converted into a coherent sentence using Natural Language Generation (NLG), encoding both categorical and numerical features in a con-

sistent, interpretable format. The analysis further explores the influence of FG by leveraging the contextual enrichment framework discussed in previous sections.

Two conditions are compared: LLM as Classifier Without FG and LLM as Classifier With FG.

In the Without FG setting, the transformation process was direct: features were rendered as simple declarative statements (e.g., “The person is male” or “The person is from Canada”), preserving the raw structure of the original data with minimal semantic augmentation. In contrast, the With FG condition employed a rule-based mapping strategy to incorporate semantically enriched features. These features, represented by scores ranging from 1 to 10, were embedded in more nuanced phrasings to capture their underlying meaning better. For instance, a cultural integration score of 1 was rendered as “the cultural integration score was low”, whereas a score of 6 became “the cultural integration score was somewhat good”. This approach ensured interpretability while preserving semantic gradations across inputs.

Each NLG-transformed sample was paired with an instruction prompt asking the LLM to classify whether the individual earns more or less than \$50,000 per year. This framing enables the LLM to perform a zero-shot classification task without any explicit parameter tuning or fine-tuning on the dataset.

A key asymmetry in the design must be acknowledged: the Without FG dataset contained the complete set of samples. In contrast, the With FG dataset was restricted to 5,000 examples due to contextual enrichment’s computational and design costs. For evaluation, traditional machine learning classifiers were trained and tested on these datasets, with a test split of 1,000 examples for the With FG case. The LLM, by contrast, was evaluated across the full 5,000 enriched samples. While this introduces a difference in test sample sizes, both conditions offer large evaluation sets to enable a meaningful comparison between zero-shot LLM classification and conventional supervised ML performance.

7.2 Textual Data

The IMDB movie reviews dataset [12] was selected for the text-based experiments. This dataset comprises 50,000 movie reviews evenly split between positive and negative sentiments, making it a well-established benchmark for binary sentiment classification tasks.

The dataset was divided into two subsets to evaluate the impact of LLM-driven augmentation:

Table 6: Experimental Configuration for Text Data.

Experiment	LLM-Augmented Dataset Size	Base Dataset Size	Performance Metrics	LLMs Used (see Table 3)	MLMs Used (see Table 1)
Contextual / Semantic Enrichment	4,000 train, 1,000 test	4,000 train, 1,000 test	Accuracy, F1, Precision, Recall	All	All
Data Synthesis	1,000 train, 10,000 test	200 train, 10,000 test	Accuracy, F1, Precision, Recall, ACS	LLaMA3.2:1B, LLaMA3.2:3B, LLaMA3.1:7B (temps 5, 10, 15)	All
LLM as Classifier	5,000 test	40,000 train, 5,000 test	Accuracy, F1, Precision, Recall	All	All
Disambiguation	3,000 train, 10,000 test	3,000 train, 10,000 test	Accuracy, F1, Precision, Recall	LLaMA3.2:1B, LLaMA3.2:3B, LLaMA3.1:7B	All
Robustness	3,000 train, 10,000 test	3,000 train, 10,000 test	Accuracy, F1, Precision, Recall	LLaMA3.2:1B, LLaMA3.2:3B, LLaMA3.1:7B	All
Compression	5,000 train, 10,000 test	5,000 train, 10,000 test	Accuracy, F1, Precision, Recall, # of Tokens	All	All

a base set and an LLM-augmented set. Both subsets were constructed to be equal in size, as much as possible, to ensure comparability. Each was split into training and testing sets using a fixed seed, typically in an 80/20 ratio. The LLM-augmented set contains all the samples from the base set, augmented or enriched per the techniques detailed in Section 4. Bag-of-Words (BOW) models were used to create learnable representations of reviews, incorporating the 10,000 most frequent terms with an n-gram range of (1,2). This setup captures individual words and short word sequences, enhancing the model’s ability to detect patterns and context.

Performance was assessed using standard classification metrics: Accuracy, Precision, Recall, and F1-score. Additionally, the Average Cosine Similarity (ACS) metric was employed in the Data Synthesis (DS) experiments to quantify the semantic similarity between synthetic and real samples. Details on the ACS metric and its computation are provided in the following subsection.

A tailored selection of LLMs was applied across the experiments, typically using default temperature settings (0.7 or 0.8) per Ollama’s configuration [10]. The DS experiment explored various temperature settings to optimize ACS values and better understand the trade-off between diversity and classification performance from generated data.

A standardized suite of machine learning models was used across base and augmented datasets to evaluate the downstream impact of the LLM-driven augmentation strategies. These models are listed in Table 1, while Table 6 provides a summarized overview of the complete experimental setup for the text domain. The following subsections outline each experiment’s specific objectives, configurations, and considerations. Key parameters for

each setup are to be cross-referenced in Table 6 for clarity.

Textual Data Flow

A flowchart is presented in Figure 2, which will be briefly discussed here to clarify the process behind the textual experiments. The pipeline begins with the full raw dataset, which undergoes minimal pre-processing, specifically, replacing a special character to denote new lines with actual newline characters. This step results in the base dataset, which is the input for multiple experimental paths.

First, the base data was used to train BOW-based ML models, which were later compared to models trained on synthetically generated reviews produced during the data synthesis experiment. Additionally, the LLM as a classifier experiment used the same base data directly, allowing the LLM to perform classification without traditional feature engineering. These LLM-based predictions were also compared against the BOW ML models.

The base data was further augmented using two distinct approaches: qualitative and non-qualitative LLM-based data augmentation. The primary distinction between them is whether the structure or correctness of the generated outputs can be verified. The qualitative augmentation focused on the contextual/semantic enrichment experiment, where the LLM generated interpretable tabular features (e.g., actor performance on a scale from 1 to 10), which could be structurally validated. In contrast, non-qualitative augmentation involved tasks such as disambiguation, robustification, and compression. Here, while the text was modified, the correctness of the changes could not be objectively verified at a structural level.

These augmentation strategies, qualitative and non-qualitative, produced separate datasets used to train BOW-based ML models. These were then compared to models trained on the original base data to evaluate the effectiveness of the augmentations.

Finally, the semantically enriched data produced through qualitative augmentation was used in two additional experiments. First, ML models were trained directly on the tabular features. Second, these tabular models were combined with text-based models in an ensemble, allowing for a performance comparison between isolated and hybrid approaches.

Data Synthesis

This experiment explores the feasibility of training BOW classifiers using entirely synthetic textual data, generated without access to real-world examples. The setup represents an extreme form of transfer learning, where the LLM functions solely as a data generator. Consequently, 1,000 reviews were generated (per temperature and model) with a 50/50 class balance; the test set here was real (non-synthetic) and identical to the 10,000-sample test set of the base dataset.

To simulate a data-scarce environment, the performance of classifiers trained on synthetic data is compared to classifiers trained on a limited real dataset containing only 200 instances, with the same 10,000-sample Base test dataset. Synthetic reviews were produced using three variants of the LLaMA model: LLaMA3.2:1B, LLaMA3.2:3B, and LLaMA3.1:7B. Each model was evaluated under temperature settings of 5, 10, and 15 to examine how temperature affects semantic diversity and classification performance.

The semantic diversity of the generated datasets was quantified using Average Cosine Similarity (ACS), where lower ACS values indicate more diverse reviews. It is hypothesized that increased semantic diversity may enhance the generalization ability of downstream classifiers.

Contextual/Semantic Enrichment

This experiment examines whether adding contextual features as a tabular data modality aids the classification performance of models. The enrichment process involved prompting an LLM to evaluate each review and assign a score along four key features:

- Plot
- Characters

- Acting & Performances
- Cinematography

Instead of altering the original review text, the LLM-generated scores were stored as additional tabular features. This design enabled two complementary approaches: independent classification using only the enriched tabular features and combined classification via an ensemble model that integrates textual and tabular modalities. When the LLM failed to produce valid or processable scores, a default score 5 was assigned to maintain dataset consistency.

The evaluation compared three classifier configurations:

1. Models from Table 1 trained exclusively on the LLM-derived tabular features.
2. A BOW classifier trained on the raw textual content of the reviews.
3. An ensemble model that fuses the predictions of the tabular and textual classifiers by averaging their class probabilities.

The approach produced 5,000 feature-enriched instances, from which BOW classifiers were trained on 4,000 and tested on the remaining 1,000 samples. Performance was compared to classifiers trained on the same, original (unenriched) training data and evaluated on the same 1,000-sample test set from the base dataset.

Disambiguation

This experiment investigates the impact of reducing textual ambiguity on classification performance. Reviews were processed using a non-qualified prompting strategy, where an LLM was asked to clarify vague or semantically unclear expressions without altering the original meaning. In this unsupervised approach, the LLM outputs were not verified or corrected: any hallucinations, deviations from instructions, or other errors remained unaddressed.

Practically, this means that on rare occasions, the LLM introduced unintended artifacts, for example, adding prefatory phrases like “Here is a 300-word review that complies with your rule set...” or, if the review contained quotes or meta-references (e.g., “As the director intended...”), hallucinating additional context not present in the original. (Example: the LLM might incorrectly insert fabricated plot points or reinterpret quotes.)

Disambiguation was performed using the following models: LLaMA3.2:1B, LLaMA3.2:3B, and

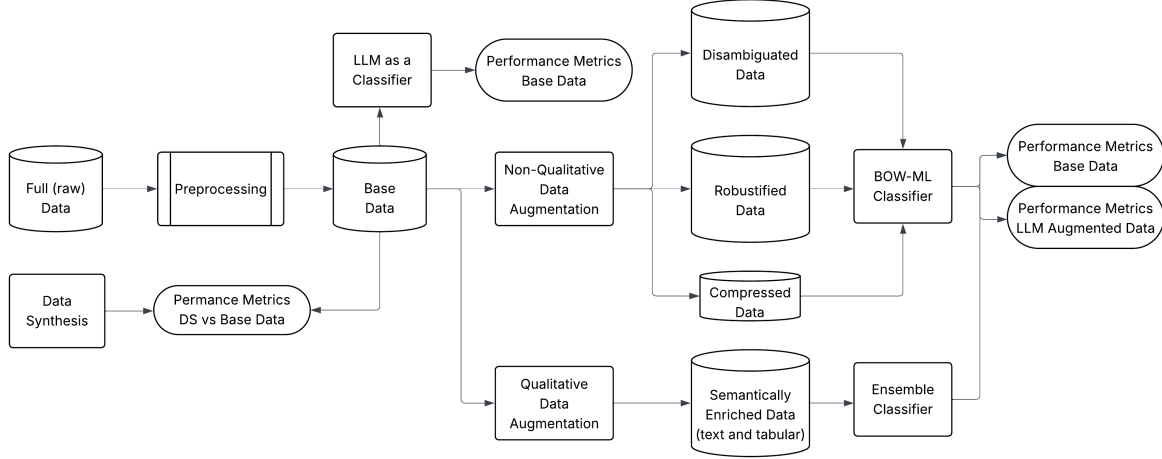


Figure 2: Textual Data Flow per LLM (Table 3).

LLaMA3.1:7B. Each LLM processed 3,000 reviews and was subsequently evaluated on 10,000 samples from the base dataset. Performance was compared to classifiers trained on the same, original (ambiguous) training data and evaluated on the same 10,000-sample (base) test set.

Robustness

This section investigates whether correcting textual noise, such as spelling errors, grammatical issues, and other language mistakes, improves classification performance. Review texts were processed using LLMs prompted to correct errors without altering the original meaning. Like the disambiguation experiment, this procedure followed an unsupervised approach, meaning that outputs were not verified. Although qualitative observations indicated a low incidence of hallucinations, occasional errors still occurred, typically as incorrect or unintended corrections (e.g., misinterpreting the original text).

The models used in this experiment were LLaMA3.2:1B, LLaMA3.2:3B, and LLaMA3.1:7B. Each model corrected 3,000 reviews and was subsequently evaluated on 10,000 samples from the base dataset. Performance was compared to classifiers trained on the same, original (non-robust) training data and evaluated on the same 10,000-sample test set.

Compression

This experiment verifies whether summarized reviews preserve enough discriminative information for on-par classification performance. LLMs were asked to write a summary for each review to retain its core content and sentiment while minimizing re-

dundancy and length. The effectiveness of the compression was calculated as the percentage reduction in the number of tokens. As in previous experiments, this unsupervised approach was occasionally prone to hallucinations, typically but rarely, manifesting as incorrect summarizations or deviations from the prompt instructions. Models were trained on 5,000 compressed samples and evaluated on 10,000 test samples from the base dataset. Performance was compared to classifiers trained on the same, original (uncompressed) training data and evaluated on the same 10,000-sample test set.

LLM as Classifier

This final experiment evaluates the ability of LLMs to perform direct classification via prompting, bypassing the need to train a separate machine learning model. Reviews were directly input into the LLM, which was instructed to output the corresponding class label for 5,000 instances. The performance of this approach was compared against BOW classifiers, trained on 40,000 samples, and tested on the same 5,000-sample test set. It is important to note that the LLMs were not fine-tuned for this task; only the default LLM was used.

7.3 Image Data

To evaluate the efficacy of LLMs in enhancing ML performance, this section investigates how the proposed benefits apply to image data. The SoCal dataset [17] serves as a representative example for this data type. It contains 15,474 images of houses, each with a resolution of 415×311 pixels. In addition to the images, the dataset includes tabular features such as square footage, number of bedrooms,

Table 7: Experimental Configuration for Image Data.

Experiment	VLLM-Augmented Dataset Size	Base Dataset Size	Performance Metrics	VLLMs/SD Used (see Table 4)	NN/MLM Used (see Table 1) and Table 2)
Contextual / Semantic Enrichment	4,000 train, 1,000 test	4,000 train, 1,000 test	MAE, $\mathbb{R}^2$	All vision models	Base NN (image+tabular) ResNet (image+tabular)
VLLM as Regressor	5,000 test	10,297 train, 5,000 test	MAE, $\mathbb{R}^2$	All vision models	Base NN (image), ResNet (image)
Compression	2,000 train, 500 test	2,000 train, 500 test	MAE, $\mathbb{R}^2$, Compression bytes	All vision models	All MLMs (BOW)
Disambiguation	2,000 train, 500 test	2,000 train, 500 test	MAE, $\mathbb{R}^2$	All vision models	Base NN (image+text+tabular), Base NN (image+tabular), ResNet (image+tabular)
Data Synthesis	1,000 train, 14,297 test	1,000 train, 14,297 test	MAE, $\mathbb{R}^2$	All diffusion models (temps 5, 10, 15)	Base NN (image), ResNet (image)
Robustness	5,000 train, 10,297 test	5,000 train, 10,297 test	MAE, $\mathbb{R}^2$	Stable Diffusion v1.5 (Strengths 0.01, 0.03, 0.06)	Base NN (image), ResNet (image)
Up-scaling	2,000 train, 500 test	2,000 train, 500 test	MAE, $\mathbb{R}^2$, Comparison LANCZOS	Real-ESRGAN	Base NN (image), ResNet (image)

and bathrooms. The image and tabular data can predict the target variable: house price.

The dataset was divided into two subsets: a base dataset and a Vision Large Language Model (VLLM)-augmented dataset. These subsets were made as similar in size as possible to ensure a fair comparison. The VLLM-augmented dataset mirrors the base dataset but includes samples modified through specific VLLM-based augmentation strategies, as described in Section 4. Both datasets are subsets of the full SoCal dataset and were divided into training and testing sets, typically using an 80/20 ratio. A fixed random seed ensured consistency across splits. Standard performance metrics were used to assess model performance, including Mean Absolute Error (MAE) and $\mathbb{R}^2$.

A tailored set of VLLMs was employed for each experiment (see Table 4, first table). The default temperature setting (0.7 or 0.8) from Ollama’s configuration [10] was generally used. In addition to the vision models, diffusion models and a Generative Adversarial Network (GAN) were employed in some experiments (see Table 4, second table). These models were incorporated to address the limitations of the Ollama vision models, which cannot edit, create, or rescale images. These capabilities were necessary for specific experiments that required additional model functionalities.

The machine learning architecture used to test the varying experimental setups consists of several Neural Network (NN) configurations, as described in Table 2. All configurations include at least an image branch in the network, which could be a simple small NN (called base NN) or a deeper network leveraging a pre-trained ResNet50 (referred

to as ResNet). Some configurations include a tabular branch: a fully connected layer that parses the house’s tabular data to base the regression on. A textual branch, which takes a VLLM-generated description of the house as input and converts it into a BOW representation, may also be used for regression. All branches converge through a concatenation layer and are processed by a final dense layer before predicting the house value. This approach can be described as a hybrid neural network (HNN) architecture, which leverages convolutional neural networks (CNNs) for image processing. Various NN configurations were tested.

For the compression experiment, however, no NN was used; instead, BOW-based ML models were employed, as outlined in Table 1.

The following subsections outline the purpose and setup of each experiment, along with relevant considerations. Readers are encouraged to refer to Table 7, which summarizes the key parameters for each experiment.

Image Data Flow

To clarify the process behind the image experiments, two flowcharts are presented in Figures 3 and 4. These diagrams correspond to two distinct data pipelines: Figure 3 illustrates the workflow using Ollama-based VLLMs, while Figure 4 represents the pipeline involving Stable Diffusion models and the Real-ESRGAN up-scaler.

The Ollama VLLM pipeline begins with the full raw dataset, including house images and associated tabular metadata. Light preprocessing is applied, specifically, ratio scaling of the tabular features and

removing several invalid images. These excluded images were either of inconsistent resolution or unrelated to the actual property (e.g., map snapshots or placeholders), likely indicating listings without proper visual data. The resulting base dataset is then used to train baseline neural networks.

Two parallel data augmentation paths are followed from this base data, similar to the textual experiments: qualitative and non-qualitative augmentations. In the contextual/semantic enrichment experiment, visual features were extracted using VLLMs and scored on a scale from 1 to 10. These scores could be structurally assessed, making this an (automatically) verifiable augmentation approach.

In the non-qualitative branch, VLLMs generated free-text (unverified) descriptions of the house images. These descriptions were merged with the original image and tabular data to create a new multimodal dataset, the merged data. This dataset was the basis for the disambiguation experiment, where neural networks were trained and evaluated against the original baselines.

The textual descriptions were also used independently to form the compressed dataset, which was input into BOW-based ML regressors. This configuration formed the compression experiment’s core and allowed performance evaluation using text-only features before integrating them into the merged dataset.

The semantically enriched dataset, generated through qualitative augmentation, was also used to train neural networks and benchmarked against the baseline models. Finally, the base dataset was used in the VLLM as a Regressor experiment, where the VLLMs themselves performed regression tasks, providing yet another point of comparison with the baseline neural networks.

Expanding on the second flowchart (Figure 4), which outlines the pipeline involving Stable Diffusion models and the Real-ESRGAN up-scaler, the same preprocessing step as previously described is applied to generate the base data. In this case, preprocessing also serves an additional purpose: developing a price probability distribution function (from the base data) used in the data synthesis experiment. Sampling from this distribution allowed the image generation of houses (with specific price points) by the Stable Diffusion models, resulting in a synthetic house image dataset.

This synthetic dataset is used to train neural networks, which are then compared to baseline models trained on the original base data. Notably, only image data is used in this comparison, with tabular features deliberately excluded to isolate the effect of image generation on model performance.

Furthermore, the base data undergoes additional transformations in alignment with other experiments. For the robustification experiment, the data is modified to decrease variance. In a separate stream, the images are upscaled using Real-ESRGAN as part of the upscaling experiment. Each of these transformations produces a distinct dataset used to train neural networks, all evaluated against the same baseline models for comparative analysis.

Contextual/Semantic Enrichment

This experiment evaluates whether adding contextual features enhances model performance in the image data domain. Specifically, Ollama vision-large language models (VLLMs) were prompted to rate house images based on the following five criteria:

- **Condition:** the general state and upkeep of the house.
- **Size:** the perceived scale or spaciousness of the property.
- **Material:** the apparent quality and type of building materials used.
- **Uniqueness:** how distinctive or architecturally uncommon the house appears.
- **Maintenance:** the cleanliness, repairs, and exterior upkeep level.

Each metric was rated on a scale from 1 to 10. A supervised prompting approach was used, where the model’s textual output was parsed into integer scores. If the model failed to produce a valid score for a given metric, a default value of five was assigned to maintain the sample. This process was applied to 5,000 house images, resulting in 4,000 training and 1,000 test samples (via an 80/20 split). A baseline dataset of equivalent size and composition was prepared without including the aforementioned contextual features for comparison.

These new tabular features, aimed at enhancing the contextual description of the images, were input alongside the initial tabular features (number of bedrooms, number of bathrooms, etc.). Now, the tabular branch in the NN consisted of 9 features instead of 4, slightly increasing the parameters for said models as well (see Figure 29 and Table 2).

VLLM as Regressor

This experiment investigated whether VLLMs could directly regress house prices from images, eliminating the need for a dedicated ML model.

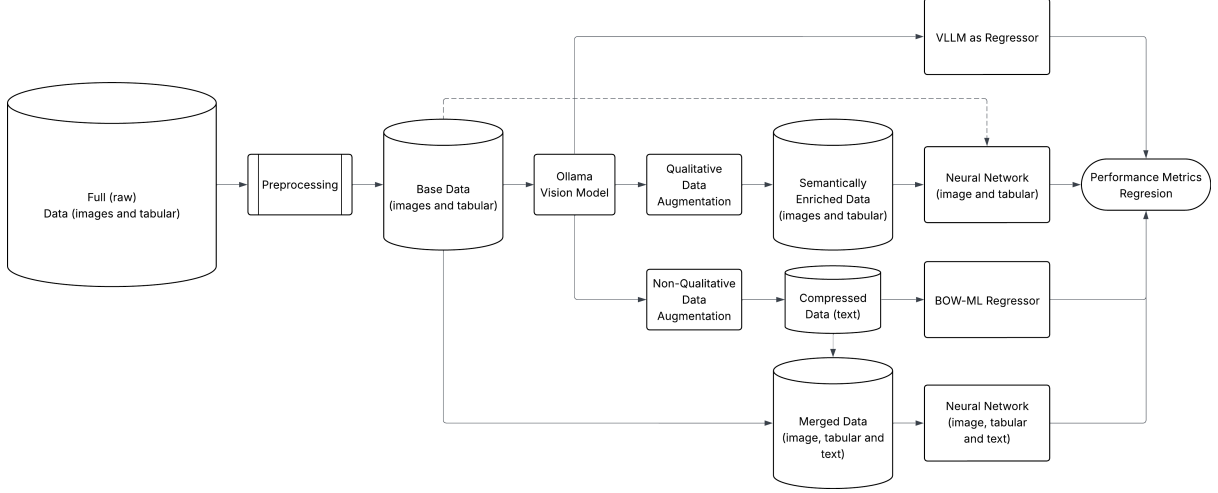


Figure 3: Image Data Flow per Ollama Vision Model (Table 4).

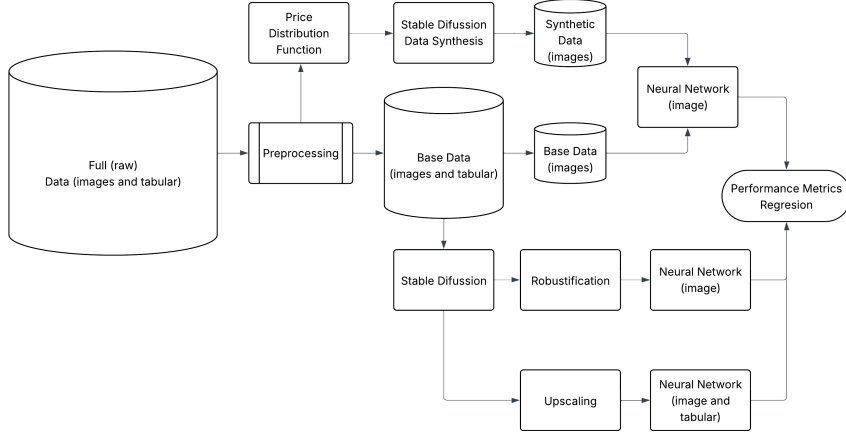


Figure 4: Image Data Flow per Stable Diffusion/GAN Model (Table 4).

Each VLLM attempted to predict the prices of 5,000 houses based solely on their image. The NNs are also evaluated on the same 5,000 images to ensure a fair comparison. The remaining dataset, comprising 10,297 samples, trains the various NN configurations.

Compression

This experiment compresses house images into textual descriptions. These descriptions are then fed to BOW regressors, and the resulting performance metrics are compared to those of a baseline NN configuration. The baseline setup parses both tabular and image data directly into the NNs. The same data partitions are used across the augmented and baseline datasets to ensure consistency.

In addition to standard performance metrics, this experiment was also evaluated based on storage efficiency. Compression was quantified in terms of bits: the total number of bits used by each im-

age was compared to the number required to store its corresponding textual description. An unsupervised strategy was used to generate the compressed text. The prompt was deliberately vague and unconstrained to encourage high variability in the descriptions, which is hypothesized to increase the discriminatory power of the BOW regressor during training. The exact prompt is documented in Section 13.6. Importantly, the prompt was instructed to avoid providing explicit price estimates to prevent the BOW model from inheriting a VLLM-derived price prediction.

However, this variability also introduced the possibility of hallucinations; for instance, the VLLM could generate details not present in the image. These hallucinations can arise because the model, operating under weakly defined prompts, may add ambiguous or missing visual information using its learned priors. These hallucinations lead to fabricated but plausible-sounding descriptions that could mislead the downstream regressor.

Note that the BOW representations were not normalized, as it was assumed that description length might encode signal relevant to house price.

2,500 samples were compressed in this manner; the Ollama vision model assigned each image a textual description. An 80/20 train-test split produced 2,000 training samples and 500 test samples. The baseline comparison used the same 2,000 training and 500 test samples but without compression (i.e., using the raw image and tabular data only).

Disambiguation

The following experiment extends the premise of the compression experiment. However, with a key difference: instead of treating the compression as a replacement, it is now incorporated as an additional input layer in the neural network to enrich the information available during training. This results in a neural network architecture with three branches: image (CNN), text (BOW), and tabular (NN). The added semantical description of the image was hypothesized to boost the performance of the NNs by providing complementary contextual cues. This setup was compared against a baseline configuration that was identical in both the size of the training and test sets but excluded the image-derived textual description, that is, it only utilized the image and tabular branches.

Data Synthesis

This experiment uses synthetic images generated by Stable Diffusion (SD) models to evaluate whether NNs can benefit from transfer learning using such data while maintaining reasonable performance. Specifically, 1,000 images were generated across three different SD models for three temperature settings, 5, 10, and 15. To generate these images, randomized prompts were created by sampling house prices from the real dataset; each sampled price was then used to prompt the SD model to create a picture of a house that visually corresponds to a property of that value.

The obtained NN regressors were evaluated using real (non-synthetic) samples. For comparison, baseline NNs were trained on 1,000 real samples and tested on the remaining 14,297 real samples. The regressors trained on synthetic data were evaluated using the same test set, ensuring controlled testing conditions across models. Notably, only image data was used to train all networks in this experiment; tabular data was deliberately excluded to isolate the effect of synthetic image generation on model performance.

Robustness

This experiment investigated whether the regression performance of NNs could be improved by “robustifying” the input images using an SD model. To achieve this, a set of 5,000 training images was “robustified” by prompting the SD model with the instruction: “Enhance core features and remove irrelevant elements to strengthen image clarity and focus.” The process used three different strength configurations: 0.01, 0.03, and 0.06, where strength refers to the degree of transformation applied to the original image.

The 5,000 robustified images were used to train the NN configurations, and, equivalently, the same set of unaltered images was used to train corresponding baseline NN configurations. Both sets of NNs were then evaluated on a standard test set of 10,297 unaltered images to obtain performance metrics. Only the image branches of the NNs were used in this experiment.

Up-scaling

The third experiment focused on image upscaling, evaluating whether a GAN-based approach, specifically Real-ESRGAN, could outperform traditional interpolation methods such as Lanczos. 2,500 compressed images were upscaled using the Lanczos interpolation method and Real-ESRGAN, each with a uniform upscaling factor $2\times$. The resulting images were then input into NN configurations utilizing only the image branches, and their performance was compared across both upscaling methods. The 2,500 samples were split into an 80/20 train-test ratio to ensure consistency, resulting in 2,000 training samples and 500 test samples for each upscaling method.

8 Results

This section analyzes the results of the experiments described in Section 7, structured by data modality (tabular, textual, and image) and the LLM-based ML strategy employed. To ensure clarity and consistency, the results are presented separately for each modality and augmentation approach, mirroring the experimental design.

The primary metric reported for both tabular and textual data is the F1-score. While other standard metrics, such as accuracy, precision, and recall, were also recorded, they have been excluded from the figures to avoid clutter. The F1-score strikes a helpful balance between precision and recall, and serves as a good overall indicator of model performance. For image-based experiments, performance is primarily evaluated using Mean Absolute

Error (MAE). MAE was chosen because it offers an intuitive interpretation: it reveals, on average, how many dollars the model is off when predicting house prices. In some cases, additional metrics provide a broader picture of how LLM-based strategies affect the data. These include non-standard metrics like Average Cosine Similarity (ACS), Average Pairwise Distance (APD), and Cumulative Explained Variance (CEV), which are introduced and explained in Section 7.

Note that most charts include a dotted line to indicate baseline performance. While the baseline is also represented as columns, the dotted line helps facilitate comparisons. In some figures concerning experiments involving five ML models, only the best and worst performing models are marked with a dotted line to reduce visual clutter.

8.1 Tabular Results

This subsection highlights the main findings from the tabular data experiments. It focuses on performance trends across different LLM-driven strategies and how these compare to baseline models. The aim is to assess whether LLM enhancements lead to meaningful improvements in classification performance.

Data Synthesis

In the data synthesis experiment, two figures are central. Figure 5 presents the F1-scores of various ML models across three conditions: the full baseline dataset, a reduced baseline with only 100 samples, and LLaMA and Mistral-generated datasets created at varying temperature settings. The results demonstrate that the models trained on both baseline versions have lower F1-scores than those trained on synthetic datasets. Also, models trained on LLaMA-generated data consistently outperform models trained on Mistral data across all temperature configurations. Additionally, performance tends to decline as temperature increases.

Figure 6 adds context by reporting error rates during data generation, which were determined by using simple heuristics, as detailed in Section 7 (subsection tabular data synthesis). While invalid samples were regenerated, the observed error rates were still recorded. Notably, LLaMA models produced higher error rates than Mistral, and error rates rose with increasing temperature. The exact figure also reports APD and CEV for each dataset. Mistral-generated datasets yielded higher APD values, while LLaMA datasets had higher CEV scores. Interestingly, temperature had little consistent effect on APD and CEV across both models.

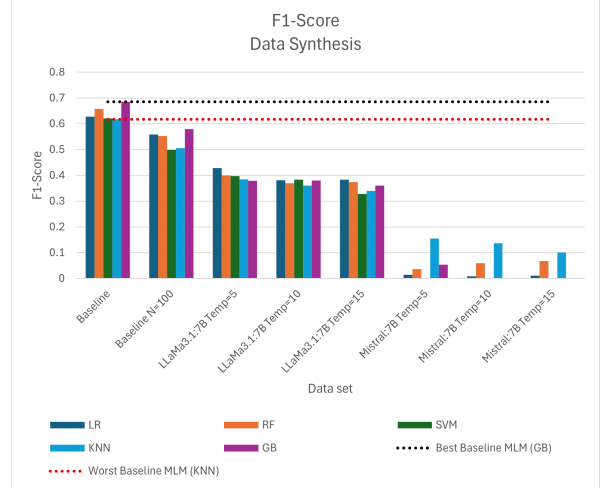


Figure 5: F1-scores of ML models on datasets generated via data synthesis. Temperature values indicate the temperature used during LLM-based data generation.

Contextual/Semantic Enrichment

The enrichment experiment, referred to as feature generation (FG), produced a dataset reused in two follow-up experiments: LLM as Classifier and Feature Selection. The main objective of this experiment was to determine whether enriching the tabular dataset with LLM-generated features would lead to improved model performance, as measured by the F1 score. Figure 7 presents the results, which exhibit a degree of variability. For instance, in the Gemma3:4B configuration, the LR model displayed a 2.3% increase in F1-score, while the KNN model yielded a 1.8% decrease under the same dataset. Overall, performance across most models remains close to baseline, with only KNN consistently exhibiting degraded performance when trained on the enriched dataset.

Interestingly, when all the LLM-generated feature datasets are merged into a single composite dataset, and models are trained on this combined data (Figure 8), the sharp drop in KNN performance is no longer observed. However, similar to the individual datasets, the overall performance remains comparable to the baseline.

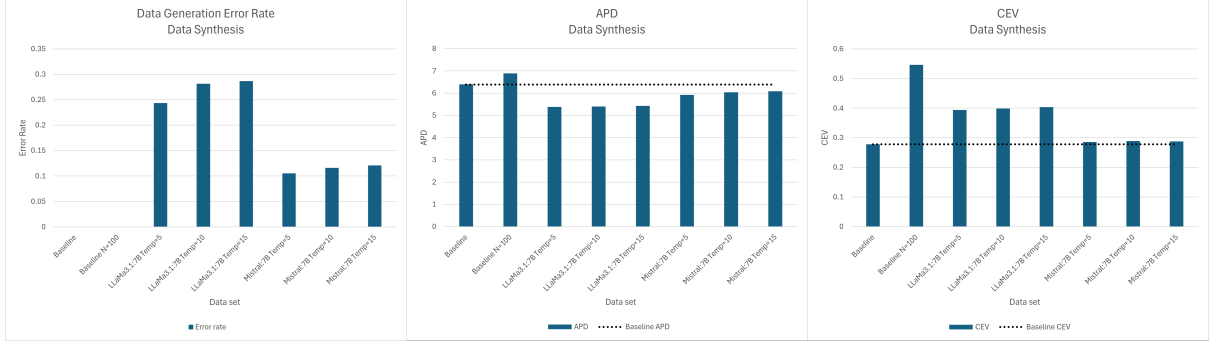


Figure 6: Error rate, APD, and CEV metrics for each generated dataset.

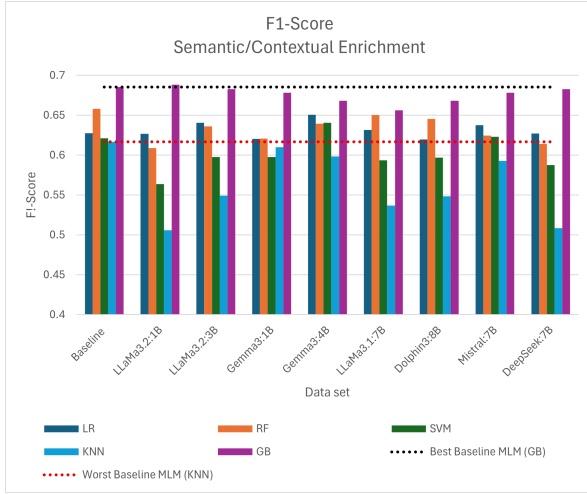


Figure 7: F1-scores of ML models trained on feature-enriched datasets.

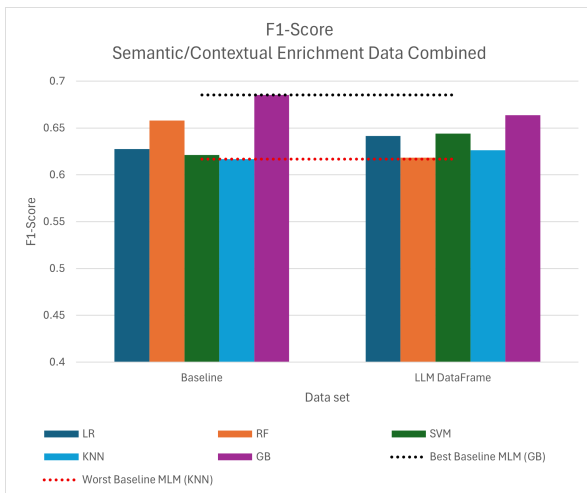


Figure 8: F1-scores of ML models trained on feature-enriched datasets combined.

Feature Selection

The Feature Selection (FS) experiment evaluates whether LLMs can effectively serve as feature selectors, under the hypothesis that, similar to domain experts, they can identify the most relevant features and their interactions to optimize model performance. This experiment also expands on the contextual/semantic enrichment experiment (feature generation, FG) by allowing FS strategies to choose from both the original and synthetically generated features. Initially, the experiment was run without the synthetic features. Figure 9 presents the accuracy of a GB model with different FS methods across feature set sizes of 5, 10, and 15. The GB model was selected due to its superior performance in preliminary benchmarks; additionally, a single model allows for more straightforward interpretability of the Figure. Similar trends were observed with the other ML models.

Several patterns are readily discernible. At the 15-feature mark, differences between FS strategies are minimal, likely because most of the key features are already included, reducing the impact of selection. However, as the number of features is reduced to 10 and then 5, the performance gaps between FS methods become more pronounced; the LLM-FS performs comparatively worse. When the synthetic features are added and FS is reapplied (Figure 10), these trends become even more visible. The performance differences between FS strategies widen, especially between LLM-FS methods.

As also mentioned in Section 7, this experiment reports when the LLM-FS method failed to produce the requested number of features (either fewer or more). These cases are referred to as “failed models”. No failures occurred in the FS without FG experimental setup, meaning feature selection on the baseline dataset yielded no failed models (none are in Figure 9).

However, in the FG condition, both Gemma3 models and the LLaMa3.1:7B model encountered

failures. The Gemma3 models exhibited severe failures, returning only the age feature across all three feature-size configurations. In contrast, LLaMa3.1:7B only failed mildly, returning 13 features instead of 15 for the 15 features configuration. These failures are reflected in the accuracy of the Gemma3 models, which drop noticeably. However, the minor failure of LLaMa3.1:7B appears to have a negligible impact on performance compared to the other models. Note that these failed models only apply to Figure 10.

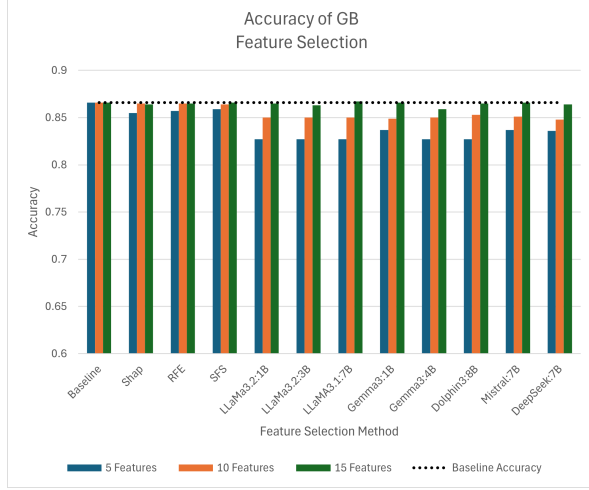


Figure 9: Accuracy of ML models trained using feature selection methods applied across three feature set sizes: 5, 10, and 15 features.

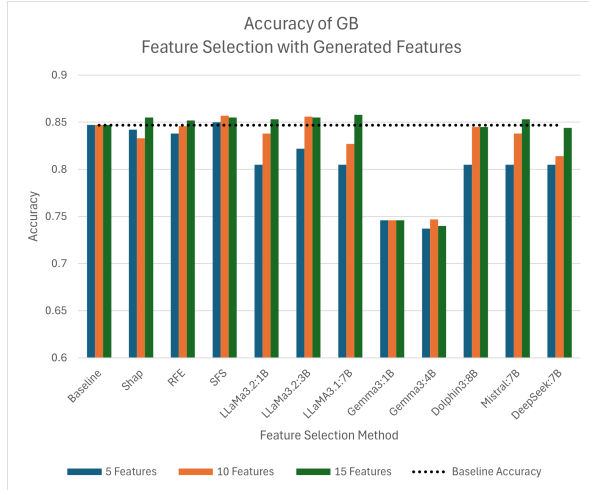


Figure 10: Accuracy of ML models trained on feature-enriched datasets with feature selection applied across three feature set sizes: 5, 10, and 15 features.

LLM as Classifier

The LLM as a classifier experiment compared the F1-scores of ML models directly against LLMs used as classifiers. Expanding on the contextual/semantic enrichment (feature generation, FG) experiment, the comparison was repeated using the synthetic features produced in the FG experiment. Figure 11 presents F1-score classification performance on the (non-synthetic) baseline data. Generally, the LLM classification performance improves as the number of model parameters increases; for example, Gemma3:1B vs. Gemma3:4B arbitrated a +29.1% improvement, and LLaMa3.2:1B vs. LLaMa3.1:7B a +3.1% gain. However, this pattern is not consistently observed across varying LLM architectures. For instance, Gemma3:4B outperformed the larger Dolphin3:8B. It is essential to note that none of the LLMs approached the performance levels of the ML models.

Including synthetic features (Figure 12) led to only minor variations in F1-scores, consistent with the earlier FG experiment. Some classifiers experienced slight improvements, while others displayed minor decreases, except for LLaMa3.2:1B, demonstrating a comparatively significant drop in F1-score.

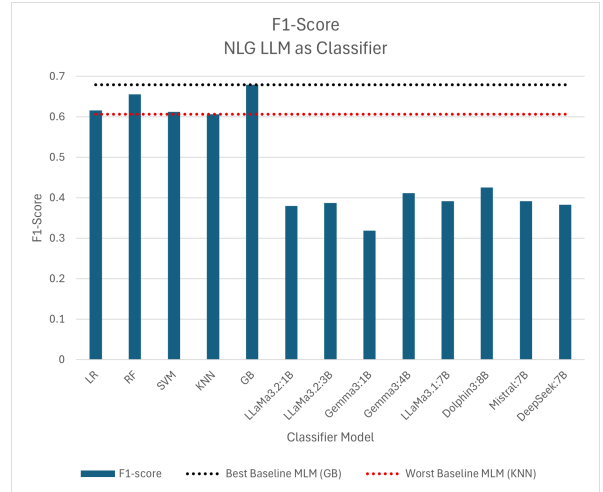


Figure 11: F1-scores of traditional ML models and LLMs used as classifiers.

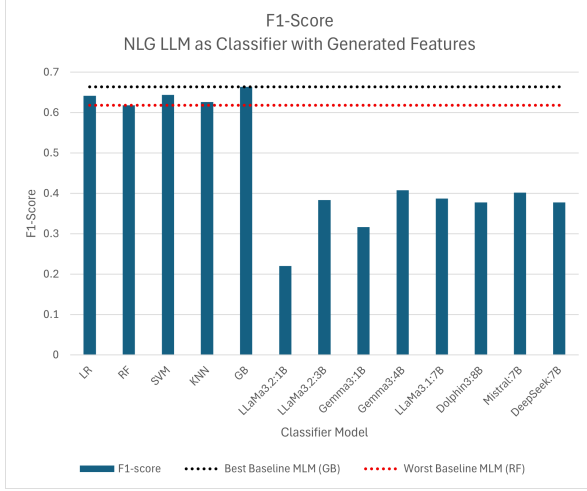


Figure 12: F1-scores of traditional ML models and LLM classifiers on feature-enriched datasets.

8.2 Textual Results

This subsection highlights the main findings from the textual data experiments. It focuses on performance trends across different LLM-driven strategies and how these compare to baseline models. The aim is to assess whether LLM enhancements lead to meaningful improvements in classification performance.

Data Synthesis

In the data synthesis experiment for textual data, artificial movie reviews were generated and used to train BOW-based ML models. The generation process was unsupervised; no alterations or refinements were applied to the reviews after they were produced. The data-generating LLMs were configured with varying temperature settings to promote diversity among the synthesized reviews. This diversity was quantified using the vectorized ACS metric, as displayed in Figure 14. Lower ACS values indicate reduced similarity among reviews, which was hypothesized to enhance the generalization performance of the downstream models.

The second figure supports this hypothesis: increasing the temperature generally led to lower ACS values. LLaMa3.2:1B corresponded with a marginal performance improvement, aligning with expectations. However, the relationship was more nuanced for LLaMa3.2:3B and LLaMa3.1:7B. Performance gains appeared between temperatures 5 and 10. Still, diminishing returns were observed at temperature 15, except for the KNN model under LLaMa3.1:7B, which interrupted this pattern.

Interestingly, LLaMa3.2:1B, despite having the lowest parameter count, yielded the highest F1-

scores across temperature settings compared to other LLM-generated datasets. Notably, this marks the first instance in which an LLM-augmented dataset outperformed the baseline, specifically for the KNN model. Moreover, the LLaMa3.2:1B results are comparable to those from the reduced baseline (200 samples), suggesting that in low-data scenarios, LLM-generated data can match or even exceed the performance of models trained on limited real data.

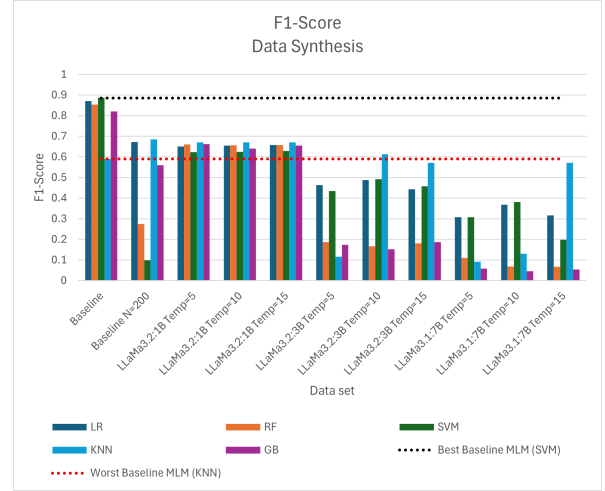


Figure 13: F1-scores of ML models trained on datasets generated via data synthesis. Temperature values indicate the temperature used during LLM-based data generation.

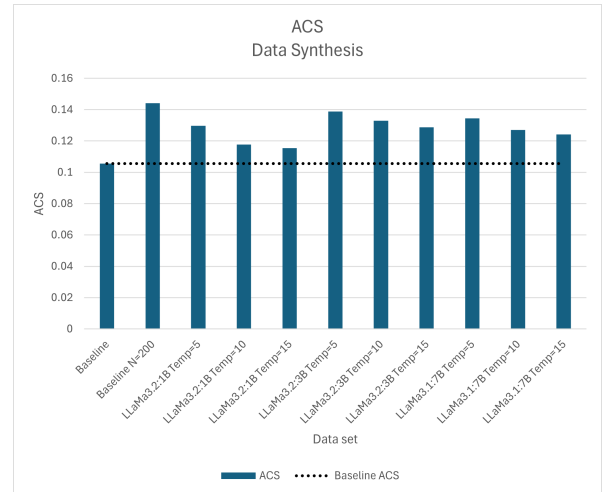


Figure 14: ACS values of datasets generated via data synthesis. Temperature values indicate the temperature used during LLM-based data generation.

Contextual/Semantic Enrichment

Contextual/Semantic Enrichment was also applied to the textual data. This experiment can be understood as a form of data transformation, converting raw textual reviews into a structured tabular format, where each column represents a specific metric, such as acting quality, plot strength, and similar attributes. The LLM assigned scores to these metrics based on the content of each review. ML models were then trained on this transformed data and evaluated using F1-scores (Figure 15).

Under this setup, the observed correlation between larger parameter counts and enhanced performance generally persists, except for the DeepSeek model. Interestingly, many ML models trained on the LLM-generated tabular datasets outperformed the baseline, where the baseline refers to the BOW ML model trained on the original textual data. The 3B parameter LLaMa3.2 model is particularly notable, as it produced datasets that consistently outperformed the baseline when used to train (all) ML models. This finding suggests that even relatively small LLMs can be leveraged through data transformation to outperform traditional BOW-based ML approaches.

Although storage reduction was not formally measured in this experiment, it's worth noting that representing reviews with just four numerical columns (ranging from 1 to 10) would significantly reduce storage requirements compared to storing the full text. In this case, the storage footprint decreases while model performance increases, a win-win trade-off.

This experiment was repeated using an ensemble approach, combining the outputs of two models: the BOW-based ML classifiers trained on the original text and the ML classifiers trained on the tabular (LLM-enriched) data. Final class predictions were determined by aggregating the prediction probabilities from each model (Figure 16). This ensemble typically improved the performance of weaker models, such as those derived from the LLaMa3.2:1B LLM. However, the ensemble did not produce significant performance gains for already strong individual models.

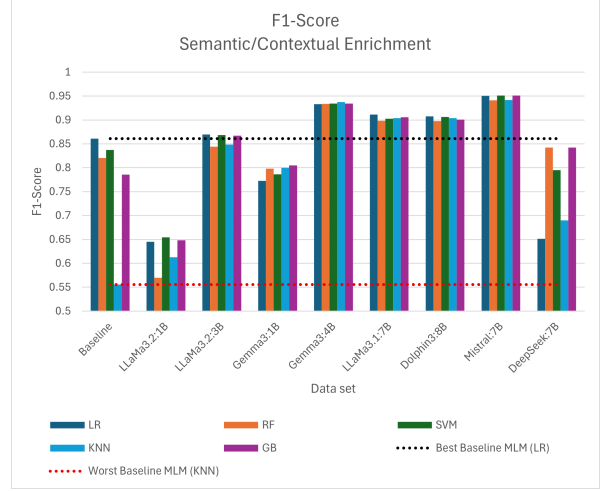


Figure 15: F1-scores of ML models trained on feature-enriched datasets.

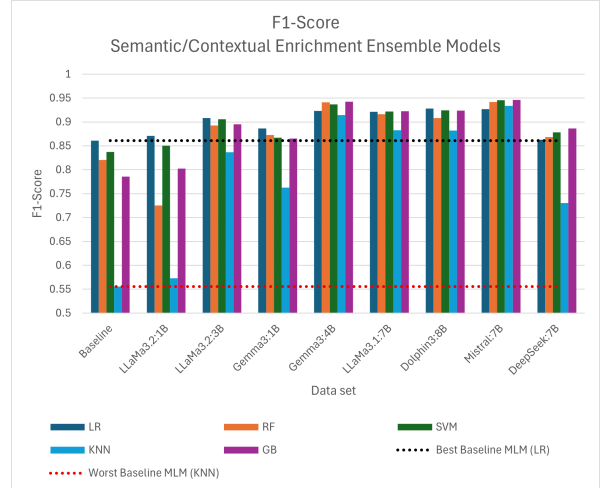


Figure 16: F1-scores of ensemble ML models trained on feature-enriched datasets.

Disambiguation

The disambiguation experiment, which aimed to reduce vague or ambiguous reviews, did not improve the F1-scores of the ML models, as displayed in Figure 17. Notably, KNN performance declines as the number of parameters in the LLM increases. Possible explanations for this trend will be explored in the Discussion (Section 9).

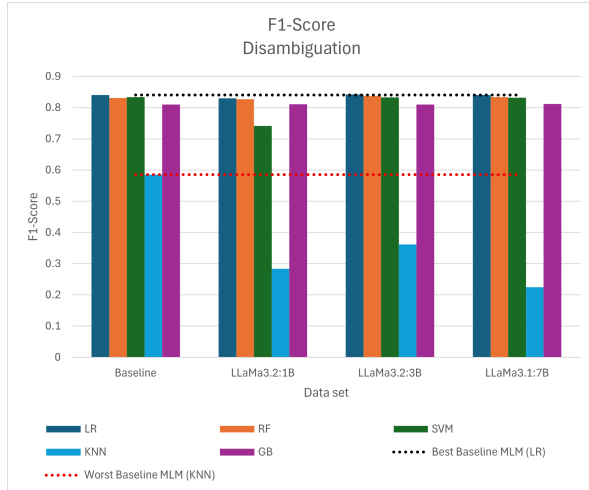


Figure 17: F1-scores of ML models trained on disambiguated datasets.

Robustness

The robustness experiment, which aimed to correct language errors in the movie reviews, did not lead to improvements in the F1-scores of the ML models, as displayed in Figure 17. Notably, KNN performance declines as the number of parameters in the LLM increases. Possible explanations for this trend will be explored in the Discussion (Section 9).

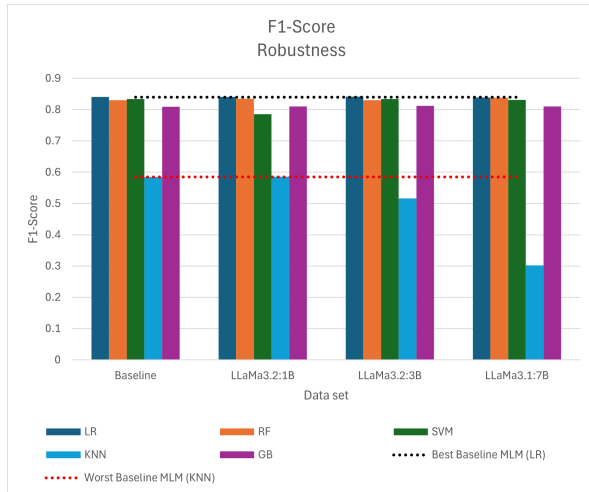


Figure 18: F1-scores of ML models trained on robustified datasets.

Compression

The textual compression experiment used LLMs to summarize movie reviews, creating compressed datasets that were then used to train BOW ML models. The objective was to examine the relationship between data reduction and classification per-

formance. As demonstrated in Figure 19, the compression process successfully reduced the number of tokens in the review text by approximately 65-80% across LLM architectures, with most achieving around 80% reduction.

Despite this substantial compression, the performance of the BOW ML models remained comparable to the baseline trained on uncompressed data. As observed in previous experiments, smaller LLMs generally performed worse in preserving F1-scores, though they were still effective in reducing data size. Interestingly, the LR model trained on data compressed by LLaMa3.2:1B yielded only a 4.4% drop in F1-score despite an 82% reduction in text length. Depending on the context of the application, this trade-off could be highly favourable.

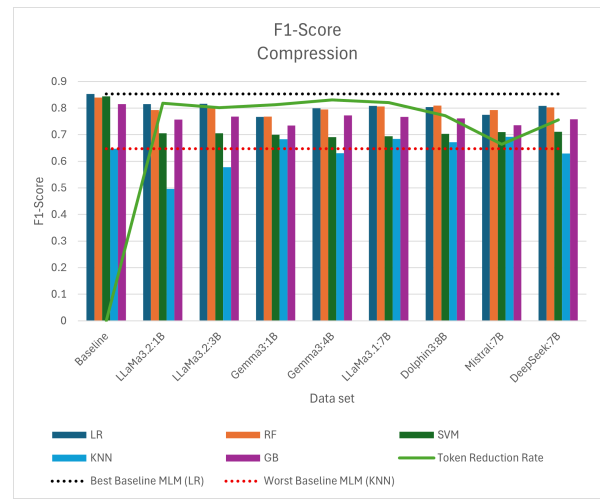


Figure 19: F1-scores of ML models trained on compressed datasets.

LLM as Classifier

The final textual experiment explored whether LLMs could be used directly to classify movie reviews as positive or negative, bypassing traditional modelling and data engineering. As illustrated in Figure 20, this approach is viable: all LLMs with more than 4B parameters outperformed the traditional BOW ML models, except DeepSeek, which did not surpass ML performance. This result also reinforces earlier observations that higher parameter counts generally lead to better classification performance within the selected LLM line-up.

Nonetheless, adopting a contrarian perspective, it is essential to acknowledge that the highest-performing LLM in its role as a classifier, Mistral:7B, achieved an F1-score of 0.943, which is lower than the best model from the earlier semantic enrichment experiment (Figure 15), where the Mistral:7B + GB BOW model achieved an F1-score

of 0.951. A relatively simple BOW model (powered by LLM data augmentation) could outperform the stand-alone LLM, suggesting that there is still meaningful value in performing traditional modelling and data engineering to maximize performance. Nevertheless, the data also indicates that one can achieve most of that performance, without any additional pipeline complexity, by simply using a 4B parameter LLM, which already outperforms standard BOW ML baselines.

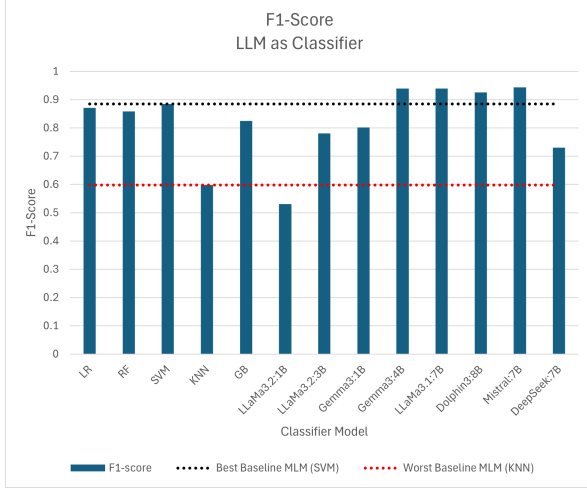


Figure 20: F1-scores of traditional (BOW) ML models and LLMs used as classifiers.

8.3 Image Results

This subsection highlights the main findings from the image data experiments. It focuses on performance trends across different VLLM and SD-driven strategies and how these compare to baseline models. The aim is to assess whether these enhancements lead to meaningful improvements in regression performance.

Contextual/Semantic Enrichment

The final contextual/semantic enrichment experiment was conducted on the image data. In this setup, various metrics, such as the condition and size of the house, were scored on a scale from 1 to 10 by VLLMs. This generated tabular data was combined with the existing structured features already provided with the image dataset, such as square footage, number of bedrooms, and other property attributes.

As previously described, the neural network architectures used in this experiment contain separate branches for processing tabular and image data, which are later concatenated in the deeper layers to produce a single regression output. Figure 21

demonstrates the MAE of the predicted values for this experiment. The baseline model used only the original tabular features, without the VLLM-generated columns.

From the results, it can be observed that the base NN model did not benefit from the semantic enrichment. The MAE slightly increased, though not significantly, suggesting that the additional features did not help or harm model performance. However, for the ResNet model, a substantial improvement in MAE was observed across all three synthetic datasets generated by the VLLMs. Interestingly, the smallest model, Gemma3:4B, outperformed the larger VLLMs in this setup, suggesting that model size alone does not determine the quality of feature generation for this task.

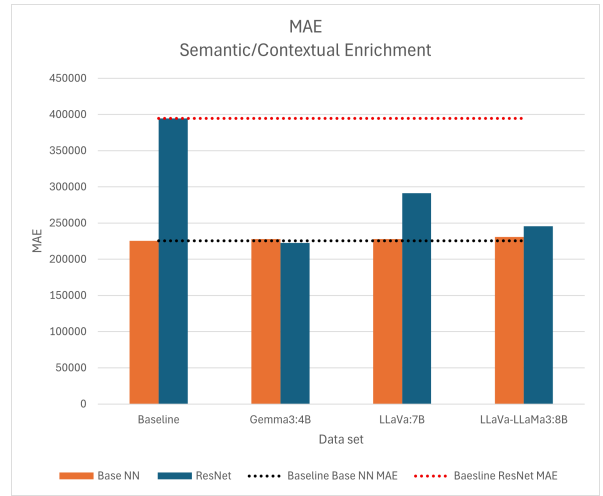


Figure 21: MAE of ML models trained on feature-enriched datasets generated by VLLMs.

VLLM as Regressor

This experiment compares the MAE of neural networks trained on the baseline dataset with the performance of VLLMs used directly as regressors, effectively bypassing the need for traditional data engineering and modelling. As demonstrated in Figure 22, the lowest scoring MAE VLLM, Gemma3:4B, achieves a lower MAE than the ResNet model (4.0% lower), but a higher MAE than the base NN (15.1% higher). Interestingly, the larger VLLM models produced comparatively higher MAE scores, suggesting that increased model size does not necessarily translate to better regression performance in this context.

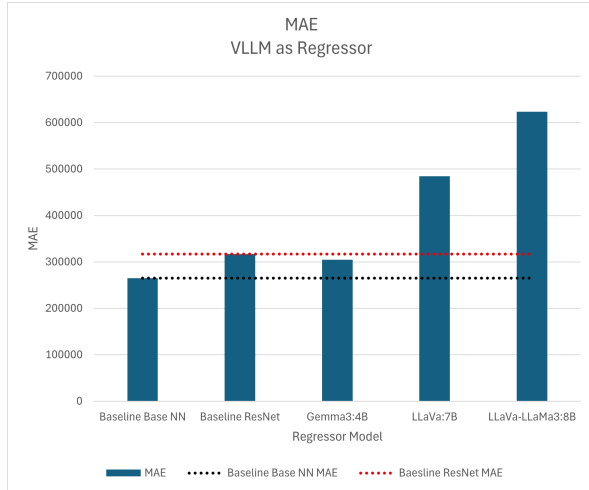


Figure 22: MAE of neural networks and VLLMs used as regressors.

Compression

The compression experiment, similar to the one conducted on textual data, aimed to examine the relationship between data reduction and regressor performance. As illustrated in Figure 23, the compression process successfully reduces the number of bytes required by approximately 95% across all VLLM architectures. This substantial decrease is achieved by transforming the data from the image domain to the text domain, where images are “converted” into textual descriptions by the VLLM. However, this reduction comes at a cost: the MAE increases by an average of roughly 24% across the neural networks trained on VLLM-augmented datasets, with all models exhibiting fairly similar MAE values in this experiment.

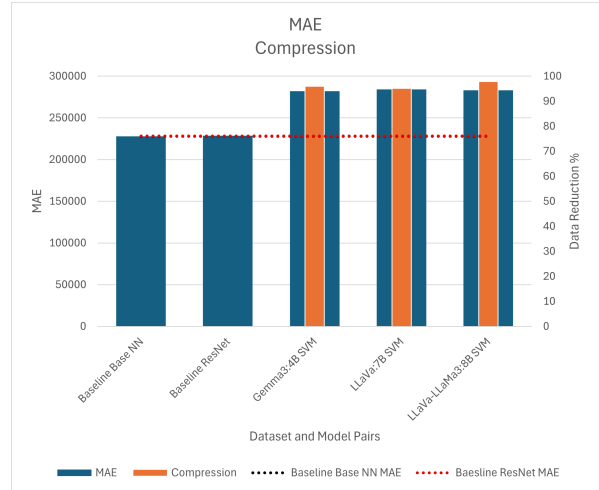


Figure 23: MAE of neural networks trained on baseline datasets compared to the best-performing ML model (SVM) trained on VLLM-compressed datasets. The secondary axis denotes the percentage reduction in data size achieved by VLLM compression.

Disambiguation

Continuing from the compression experiment, the disambiguation experiment leverages the textual descriptions generated by the VLLMs by reintegrating them into the neural network architecture via a third branch. This additional textual branch uses a BOW approach to learn house values, which is then merged with the image and tabular branches through a concatenation layer to predict the final house price. As illustrated in Figure 24, this strategy did not result in lower MAE scores for any of the neural networks trained on the VLLM-generated dataset. The proportionality of the MAE values is similar to those observed in the earlier VLLM as regressor experiment, once more indicating that higher parameter counts do not necessarily correlate with lower MAE performance.

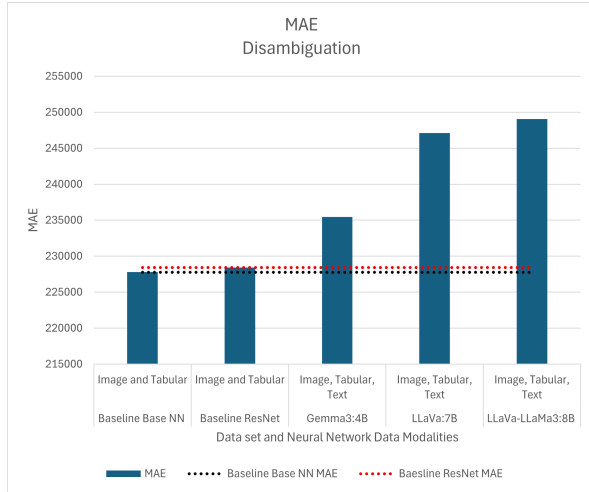


Figure 24: MAE of neural networks trained on baseline datasets (image and tabular modalities) versus those trained on disambiguated datasets (image, tabular, and text modalities).

Data Synthesis

The data synthesis experiment employed a series of Stable Diffusion (SD) models to generate synthetic images of houses. The process began by constructing a price probability distribution from the original housing dataset, which was then used to sample target values for image generation. For instance, if houses priced at \$300,000 accounted for 12% of the dataset, there was a corresponding 12% likelihood that the generation prompt would request an image of a house at that price. Sampling was performed as a continuous distribution, allowing values to be selected down to the dollar level.

As in the data synthesis experiments for other modalities, different temperature settings were tested to promote image diversity, hypothesizing that greater variation would lead to better generalization performance. Interestingly, for the Base NN, the synthetic datasets performed comparably to the baseline (Figure 25). In particular, the SD-2-1 model with temperature 5 produced results with only 2.2% higher MAE than the baseline. However, for the ResNet models, there was more volatility in MAE, even between runs using different temperatures with the same diffusion model. That said, some ResNets trained on synthetic data outperformed the baseline. For instance, SD-2-1 yielded a 3.4% lower MAE than the baseline ResNet. The potential reasons behind this volatility will be discussed further (Section 9).

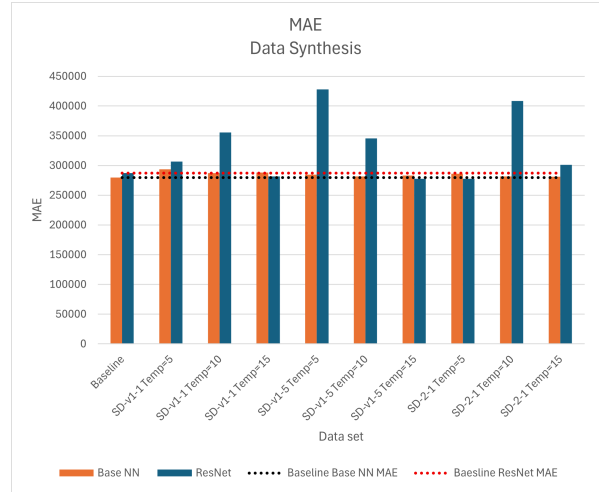


Figure 25: MAE of neural networks trained on datasets generated via VLLM-driven data synthesis. Temperature values indicate the temperature used during VLLM-based data generation.

Robustness

The robustness experiment examined how altering training images influenced MAE when training neural networks on the augmented data. The experiment was controlled by a single parameter, strength, which determined the intensity of image augmentation applied via the Stable Diffusion (SD) model. Three strength levels were tested: 0.01, 0.03, and 0.06. Modifying the images was to “prime” them for training by potentially removing or simplifying visual elements that contribute little to predicting house value. For instance, blurry or unfocused objects, such as an indistinct bush, might be transformed to appear with sharper edges. This kind of visual clarification could help a CNN better recognize features indicative of a home’s condition, for example, the state of the landscaping as a proxy for overall maintenance, which may influence value estimation.

As demonstrated in Figure 26, the lowest strength setting (0.01) resulted in an MAE for the NN base model nearly identical to the one trained on the unaltered baseline dataset. However, training on the dataset augmented at this strength for the ResNet model led to a 9.3% reduction in MAE. In contrast, models trained on datasets altered with higher strength values (0.03 and 0.06) did not outperform either the baseline or the 0.01 strength configuration.

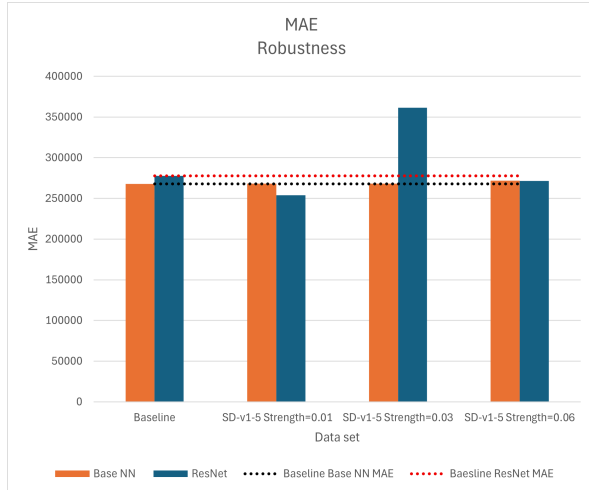


Figure 26: MAE of neural networks trained on robustified datasets. Strength denotes the intensity of augmentation applied to images, with higher values corresponding to stronger augmentation.

Up-scaling

The final image experiment investigated whether upscaling images using the transformer-based GAN, RealESRGAN, could improve model performance, as measured by MAE. Previous studies have reported that image upscaling can enhance downstream classifier performance. To evaluate the effectiveness of RealESRGAN, its results were compared to LANCZOS, a traditional interpolation-based upscaling method. As illustrated in Figure 27, upscaling had little impact on the performance of the base NN model. However, for the ResNet model, both upscaling techniques led to reduced MAE, with LANCZOS having a 59.7% lower MAE than RealESRGAN.

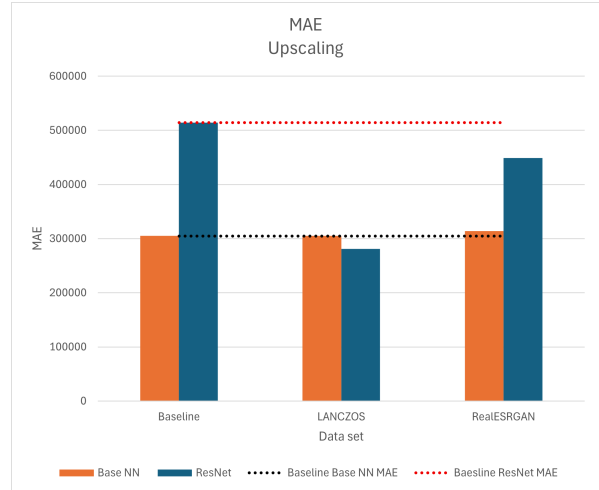


Figure 27: MAE comparison between neural networks trained on baseline and upscaled datasets.

9 Discussion

This work explored the integration of LLMs and other transformer-based architectures into machine learning pipelines, aiming to enhance predictive performance. A structured framework was proposed to formalize this process, offering guiding principles for implementing LLM-based enhancements. A series of experiments was conducted and detailed with supporting figures and tables. Results were primarily evaluated using F1-scores and MAE, alongside additional metrics for a more comprehensive analysis. While the Results section highlighted key findings and preliminary interpretations, this Discussion aims to synthesize those outcomes, explaining why these results emerged and what they signify in the broader context of the proposed framework and the practical viability of LLM integration into end-to-end ML workflows.

9.1 Overview of Data Modalities

Before examining specific experiments, this discussion begins at a high level by identifying where the framework’s application is most justified, based on the experimental results. Notably, tabular data appeared to benefit the least from the applied template. Across many experiments, LLM-enhanced models did not demonstrate consistent or competitive performance improvements. While there were exceptions, such as the semantic/contextual enrichment experiment, where the LLM-augmented Logistic Regression model outperformed its baseline counterpart (see Figure 7), these were not the norm. In general, baseline-trained models performed better. This observation was particularly evident in

the LLM-as-classifier, data synthesis, and, to some extent, the feature selection experiments, which exhibited moderately large performance gaps. This outcome is unsurprising, as LLMs typically struggle more with numerical tasks than linguistic ones [19].

In contrast, applying the framework to textual data yielded far more promising results. This finding aligns well with expectations and existing literature. The findings suggest that using the framework for NLP tasks can provide substantial performance gains and non-performance benefits, such as data compression. Practitioners have multiple viable paths, whether through synthetic data generation to bootstrap models in low-resource settings, contextual feature enrichment, or even direct use of LLMs as classifiers. While the framework was not without its shortcomings, specifically, the robustification and disambiguation experiments did not result in significant performance improvements, these limitations will be examined in more detail later.

Lastly, the image data experiments produced a set of intriguing outcomes. While it is challenging to derive definitive conclusions regarding the framework’s effectiveness for this modality, several results indicate its potential. Notably, the data synthesis approach yielded performance on par with baseline-trained models. More strikingly, the VLLM-as-regressor model outperformed a deep learning baseline, although it was still surpassed by a shallower neural network configuration by approximately 10% reduced MAE. Additionally, while the data reduction experiment achieved substantial compression, it came at the cost of downstream accuracy, suggesting that future work could focus on improving retention mechanisms. Interestingly, unlike in the textual experiments, robustification aided the training of the deep neural network, though fine-tuning the strength parameter may yield even better results. However, the up-scaling and disambiguation experiments did not surpass traditional methods like LANCZOS or the baseline models in performance; these findings will be investigated later.

9.2 Interpreting Unexpected Results

This section examines individual experiments, focusing on those whose outcomes diverged from the expectations set by the proposed framework in Section 4. By analyzing these deviations, a deeper understanding of the limitations and edge cases of LLM integration in machine learning pipelines is pursued.

Tabular Data Synthesis

In the tabular data synthesis experiment, models trained on LLM-generated synthetic data did not outperform those trained on the baseline or even on scarce data configurations. This finding suggests that, for structured (tabular) datasets, LLMs may struggle to generate samples that preserve the statistical properties and relationships critical for predictive modeling.

Additionally, higher LLM temperatures correlated with a decline in average model performance. This drop may be due to the increased randomness introduced at higher temperatures, which, rather than enhancing meaningful variation, appears to inject noise into the generated data. Figure 6 supports this finding, as the first graph indicates that the error rates, as measured by simple validation heuristics, increase with temperature. Moreover, APD and CEV remain stable across temperature settings, suggesting that despite the drop in sample validity, there is no meaningful increase in data diversity (as measured by APD) or internal data complexity (as reflected by CEV). However, it is worth noting that the APD and CEV metrics may lack the granularity to capture certain types of sample variance, meaning subtle or high-dimensional diversity could still be present but undetected by these metrics. In summary, the findings suggest that increasing the temperature for tabular data generation by LLMs does not lead to greater sample variance but instead increases sample invalidity. However, more granular metrics may be required to confirm the absence of increased variance conclusively.

Another noteworthy finding was that the LLaMa model consistently outperformed the Mistral model in the tabular data synthesis task. This performance gap may be attributed to differences in pretraining data and/or architectural design, which could make LLaMa more appropriate for modelling structured numerical relationships. Mistral models may lack the inductive biases necessary to preserve relational consistency in synthetic tabular data. This result underscores the importance of model selection in LLM integration, as significant variation in task-specific performance can occur even among models with similar parameter scales. This is not just an isolated finding; many experimental results demonstrate considerable variance between LLM architectures.

Tabular Contextual/Semantic Enrichment

Several factors may explain why the generated features in this experiment did not lead to meaningful improvements in performance metrics. One possibility is that the hypothesized features failed to introduce sufficient general world knowledge into

the dataset. Many enriched features were derived through inference from existing tabular fields, meaning they did not introduce truly novel or external information. This theory suggests that inferring features solely from the existing data may be insufficient to improve predictive performance in tabular contexts, contrary to the improvements observed in the corresponding textual experiments.

These findings imply that more advanced strategies may be required to incorporate domain knowledge or general world understanding into structured data. For example, leveraging external data sources via Retrieval-Augmented Generation (RAG) and integrating domain-specific datasets could provide a richer foundation for generating contextually valuable features. Another possibility is that the specific characteristics of the domain used in this experiment were inherently less susceptible to enrichment, limiting the effectiveness of semantic augmentation regardless of the technique applied. Finally, it’s essential to recognize that model drift also affects LLMs. If an LLM is trained on outdated datasets, even just a few years old, its knowledge and world-view may no longer accurately reflect current information.

Feature Selection and LLM as Classifier

The findings of these two experiments are presented together, as they appear to share a common underlying limitation. Current LLMs may lack the statistical rigour and inductive bias required to outperform traditional ML and FS methods, particularly in tabular data settings. The underperformance observed in feature selection and LLM-as-classifier experiments suggests that LLMs, particularly smaller models, struggle to capture the nuanced relationships necessary for effective modelling. Given these results, practitioners are advised to exercise caution when replicating statistically grounded methods with small-scale LLMs for tabular tasks, as their performance may not meet the standards set by established techniques.

Textual Data Synthesis

As noted in the discussion of tabular data synthesis, the choice of LLM architecture plays a crucial role in downstream model performance. The results from the textual data synthesis experiment further reinforce this point: significant performance variation was observed across different LLM architectures, even among models of similar parameter size. This finding suggests that ML practitioners integrating LLMs into their pipelines should not assume consistent performance across models. Instead, careful model selection and benchmarking are essential, especially when working with smaller

LLMs, as architectural and training differences can substantially affect the quality of synthesized data.

Textual Robustness and Disambiguation

The results of the robustness and disambiguation experiments did not yield improvements in model performance and, in the case of the KNN model, even revealed performance declines. The interaction between data augmentation techniques and distance-based classification can explain these drops. More specific or detailed descriptions may have disrupted existing clusters by increasing distances between formerly similar samples in the disambiguation experiment. For instance, replacing ambiguous phrases with more explicit alternatives can introduce vocabulary that shifts samples away from their original neighbors in feature space. Similarly, in the robustness experiment, augmentations such as rephrasing or paraphrasing likely introduced new or rare words into the data. When vectorized using TF-IDF, these rare terms can disrupt the feature overlap among same-class instances, thereby degrading the meaningfulness of the distance metrics that KNN depends on. Moreover, these problems are likely amplified by the relatively small dataset sizes used in these experiments. In low-data regimes, even minor shifts in feature representation can lead to significant disruptions in model behaviour. Assuming the validity of these explanations, it suggests that textual robustness and disambiguation techniques may only become beneficial in high-data regimes, where augmentation does not significantly fragment local structure. However, this hypothesis would require testing with larger datasets to validate.

Image Experiments - ResNet Performance

As observed in the results section, the ResNet model exhibited significant variability in performance. In some experiments, it outperformed the Base NN by several percentage points; in others, it suffered from nearly double the MAE. This volatility likely stems from ResNet’s architectural characteristics. Unlike the Base NN, a relatively shallow model with a limited number of layers, ResNet contains many latent layers capable of capturing more complex and hierarchical image features. While this depth can provide an advantage in modelling sophisticated visual features, it also increases the risk of overfitting, especially in experiments with limited sample sizes.

In contrast, the Base NN demonstrated more stable performance across experiments. Its lower capacity may have limited its peak performance, but also made it more robust to variations in the input quality of the images. ResNet’s flexibility,

on the other hand, came at the cost of increased sensitivity to data characteristics.

Interestingly, specific LLM-based enhancements, such as upscaling and semantic/contextual enrichment, stabilized ResNet’s performance. This finding suggests that such enhancements may help reduce overfitting by regularizing the feature space, potentially making it easier for deeper models to learn generalizable patterns. However, it is worth noting that in the upscaling experiment, the traditional LANCZOS method still outperformed the LLM-based approach in reducing ResNet’s MAE. Supporting this interpretation is the performance observed in the image synthesis experiment. At certain temperature settings, the ResNet’s MAE increased sharply. This spike is likely due to the natural randomness in generating image data by Stable Diffusion models. As a result, the ResNet may have overfitted to the fluctuation in visual features, leading to elevated MAE values.

Image Contextual/Semantic Enrichment

A plausible explanation for the absence of a meaningful reduction in MAE from the generated visual features is that the convolutional layers of the CNN may have already captured these features. Alternatively, the enriched features might not have exhibited a strong enough statistical correlation with the target variable (house value) to contribute additional predictive power. Furthermore, the visual assessments generated by the VLLM may have been applied inconsistently across samples. However, given the reduction in MAE for the ResNet model, it appears most likely that the CNN already encapsulated the visual features, which reduced the effect of assimilating novel contextual understanding to the data.

Image Compression

This section briefly addresses the observed performance loss in the models trained on compressed data. While some degradation in performance was expected due to the significant 95% data reduction, the extent of the drop may be too severe to justify the trade-off. One likely explanation is that the textual descriptions, although prompted to be detailed, failed to fully capture the visual subtleties and context that contribute to a house’s value; after all, a picture is worth a thousand words. Inconsistencies in description generation may also have played a role. For example, the VLLM may have described multiple houses as “big” without differentiating meaningful differences in scale, leading to a deterioration of the predictive signal.

Additionally, using BOW models to process the

compressed textual data may have further limited performance. By design, BOW models discard word order and context factors that could be critical in accurately interpreting property descriptions. A more appropriate alternative might have been a Recurrent Neural Network (RNN), which is better suited to capture sequential context in language than BOW models. To more thoroughly evaluate the viability of this compression approach, future work should consider replicating the experiment using more advanced NLP architectures and evaluating whether better performance retention can be achieved.

Image Robustness and Disambiguation

The robustness experiment successfully produced a ResNet model that outperformed both models trained on the baseline data. However, the effectiveness of the transformation appears to be sensitive to the strength parameter, which likely needs to be optimized per dataset due to factors like varying resolution.

In contrast, the disambiguation experiment yielded less promising results. Compared to the compression experiment, where MAE values remained around 280,000, the inclusion of textual descriptions may have degraded performance. This finding suggests that incorporating textual data alongside image and tabular inputs in multimodal neural networks does not necessarily improve results. The underlying reasons for this are addressed in the previous subsection on the image compression experiment.

Image Up-scaling

LANCZOS outperformed Real-ESRGAN upscaling, likely because it preserves the original image content better. Unlike GAN-based methods such as Real-ESRGAN, which can introduce distortions and hallucinated features, LANCZOS applies a mathematical interpolation that maintains image integrity without adding noise (through adding content). Furthermore, Real-ESRGAN is trained on generic image datasets and was not fine-tuned for upscaling house images in this work. For predictive modeling, the fidelity and consistency of the image appear to be more critical; the simpler and more reliable Lanczos upscaling method likely provided more meaningful inputs to the neural network models.

Parameter Count and Performance

As noted in previous subsections, task-specific performance among LLMs and VLLMs varies signif-

icantly. This section extends that observation by emphasizing that parameter count does not exhibit a linear relationship with task performance (according to the data of this work). While smaller models, such as those with around 1 billion parameters, typically yielded lower performance across many experiments, this pattern was inconsistent. For instance, in the textual data synthesis experiment (see Figure 13), some smaller models performed comparably to, or even outperformed, larger counterparts.

This finding is essential for practitioners: parameter count alone cannot predict task-specific effectiveness. Architectural design and pretraining data substantially shape a model’s utility. As a result, selecting the optimal LLM for a given task remains a non-trivial challenge. This variability complicates the practical adoption of the proposed framework and its potential derivatives, as it necessitates careful benchmarking before committing to a particular model for integration.

10 Limitations

Despite the breadth and ambition of this work, several limitations should be acknowledged:

1. **Lack of Model Optimization:** All traditional ML models and neural networks used for predictions were run with their default configurations, with no hyperparameter tuning performed. While this design choice ensured reproducibility and neutral baseline comparisons, it may understate or overstate the utility of LLM-based enhancements in real-world settings, where models are typically calibrated for optimal performance. The relative benefits observed here may not generalize to predictive models with optimized parameters.
2. **Limited Exploration of Larger LLMs Due to Computational Constraints:** The work focused on small to mid-sized LLMs to evaluate whether meaningful improvements could be achieved with models accessible to most practitioners. While this approach ensures practicality, it inherently restricts insights into how more capable, larger-scale LLMs might perform within the same framework.
3. **Hardware and Time Constraints:** All experiments were performed on non-specialized, mid-range hardware. This constraint severely impacted runtime; the full suite of experiments took nearly 60 days to complete. As a result, dataset sizes, particularly for image-based experiments, had to be limited. While

care was taken to ensure that sample sizes were still large enough to yield meaningful results, some experiments, especially those involving ResNet, might have benefited from a larger dataset partitioning.

4. **Limited Modal and Domain Coverage:** Empirical experiments were limited to three data modalities: tabular, textual, and image data. While the framework was designed to be extensible to other types, as outlined in Section 5 (subsection Other data types and the template), this generalizability was not empirically validated. Furthermore, only one dataset per modality was used. This limitation restricts the generalization of findings across domains, as real-world datasets often exhibit unique characteristics that may interact differently with the proposed enhancements.
5. **Prompt Sensitivity and Lack of Systematic Prompt Evaluation:** One major limitation is the lack of formal evaluation or control over prompt variability. Prompt engineering was conducted manually and qualitatively, and even small changes in phrasing can lead to significantly different outcomes. This variability was neither quantified nor systematically analyzed, leaving potential performance variance unaccounted for.
6. **Exclusion of LLM Fine-Tuning:** Only off-the-shelf LLMs were used throughout this work. No fine-tuning was performed, despite the potential for domain-specific adaptation to improve the quality of LLM-driven enhancements, as fine-tuned LLMs may better distinguish nuances within tasks. While it is reasonable to assume that fine-tuning would strengthen the results observed here, this remains an untested assumption.

Note that while this point appears similar to point one in this section, it has a distinguishable characteristic: although both points comment on the impact of model performance within the context of the framework, the first point highlights the effect of tuning model hyperparameters on the framework, such as the maximum depth of branches in random forests, whereas the second point suggests that there are potential performance benefits to the final prediction models through the improvement of LLM-driven enhancements (by fine-tuning said LLM). For instance, fine-tuned LLMs could yield more effective feature synthesis, resulting in better features that ultimately improve the performance of the predictive models trained on them.

7. **Non-Comprehensive Framework Scope:** Although the proposed framework covers many enhancement strategies, it is not exhaustive. Some additional augmentation avenues, mentioned briefly in Section 4, remain unexplored due to time and scope constraints. While the aim was to provide a practical and modular starting point, the framework does not represent a fully comprehensive LLM enhancement strategy that encapsulates an increasingly growing library of LLM strategies.

11 Future Work

This section outlines potential research directions that could extend and refine the work presented in this paper. Many of the suggestions below expand directly on the limitations discussed in Section 10 and offer practical opportunities for expanding the proposed framework.

1. **Expanding the Framework with Additional Enhancement Modules:** Future work could explore additional LLM-driven augmentation strategies not covered in this paper. Existing literature offers a variety of potential mechanisms, such as hyperparameter tuning assistance or active learning, that could be integrated into the framework. Moreover, novel applications of LLMs to improve or automate other stages of the ML pipeline could also be pursued, as multiple undiscovered applications may be possible.
2. **Extending the Framework to More Data Modalities and Domains:** This work was limited to tabular, textual, and image data. Future research could empirically test the framework’s applicability to other common modalities, as already mentioned in this work. Furthermore, applying the framework to broader domains, such as healthcare and finance, would help solidify its generalizability.
3. **Evaluating Larger and More Diverse LLMs:** Only small to mid-sized models were used in this work. Future studies could incorporate larger-scale models to determine whether increased model size yields more substantial or consistent enhancement results.
4. **Systematic Prompt Engineering and Prompt Variance Quantification:** Prompt sensitivity was a notable limitation in this work. A valuable next step would be to develop structured methodologies for generating and refining prompts. Furthermore, quantifying prompt-induced variability

could offer more explicit confidence bounds on LLM performance.

5. **Incorporating Fine-Tuning of LLMs:** While this work relied exclusively on off-the-shelf LLMs, future research could assess how fine-tuning LLMs on task or domain-specific data affects the effectiveness of the framework’s enhancement modules. Fine-tuned LLMs may yield better enhancements, such as better feature synthesis, which ultimately improves the training of downstream models.
6. **Testing the Framework with Optimized (baseline) ML Models:** This work intentionally avoided hyperparameter tuning or architecture optimization of baseline predictive models to isolate the impact of LLM-based enhancements. A crucial next step is to evaluate how these enhancements apply to parameter-optimized models. This evaluation would help determine whether LLM integration remains beneficial when predictive models are already near their peak performance.

12 Conclusion

This work aimed to investigate the role of LLMs in optimizing various components of the ML pipeline, with the primary aim of improving predictive performance. While performance improvements were the main focus, non-performance enhancements were also considered. The research was structured around the following central questions:

1. **How can LLMs and transformer-based architectures enhance the performance of machine learning?** The experimental results illustrate that several components of the proposed framework contributed to improved ML performance, particularly through semantic and contextual enrichment, which proved effective for textual and image-based data. Additionally, using (V)LLMs directly as models for these data types demonstrated competitive results. Data synthesis via LLMs and diffusion models also yielded performance gains in low-data or transfer learning contexts. However, the techniques related to robustification and disambiguation did not demonstrate measurable benefits in the experimental setups used. These strategies may still be effective in other scenarios with different data characteristics or tasks.
 - **Can LLM-driven feature engineering improve model performance?** Yes, for textual and image modalities.

However, this work did not indicate improvements for tabular data using this approach.

- **Can synthetic data generated by an LLM aid in transfer learning and generalize well to other datasets?**

The results suggest that LLMs and Stable Diffusion models can be valuable tools for generating synthetic data in transfer learning scenarios.

- **Can LLM-based feature selection outperform traditional techniques?**

Based on the results from this work, this was not the case for tabular data. LLMs may lack the statistical rigor required to capture complex feature interactions.

- **Can data augmentation using LLMs improve performance?**

While LLM-based augmentation can offer improvements, based on the experimental results, traditional augmentation methods often remain more effective and reliable.

- **Could LLMs outperform traditional ML models?**

In some cases, yes. For example, LLMs used as classifiers for textual data and VLLMs as regressors demonstrated performance on par with, or even better than, traditional models.

- **What novel strategies can LLMs introduce to enhance model performance?**

This work presents a variety of such strategies in the methodology section (Section 4) and reviews others in the literature (Section 3).

2. **How should LLMs and transformers be cohesively integrated into data-driven decision-making and ML pipelines?**

ML practitioners can utilize frameworks like Ollama and Huggingface, which offer LLM access through API-style interfaces. These tools can be integrated into existing pipelines or used to develop new ones from scratch. However, more standardized, off-the-shelf modules that encapsulate the enhancements proposed in this work are still lacking.

This research demonstrates that LLMs can offer meaningful and sometimes competitive enhancements across several parts of the ML pipeline, particularly for image and textual data. While results for tabular data were less encouraging, the overall findings suggest that LLM-based methods are a promising addition to the ML toolkit. The proposed framework provides a starting point for prac-

titioners to reason about and implement such enhancements, though it is not exhaustive.

Crucially, these results were achieved using relatively lightweight LLMs on non-specialized hardware, highlighting the growing accessibility and practicality of these models. Their general-purpose nature opens up many applications for improving ML workflows. This work aimed to contribute a more concrete, reusable framework that future ML engineers can extend on.

Nevertheless, the broad scope of the work caused some limitations. These include limited evidence for generalizability across data modalities and domains, the exclusion of optimized models in testing LLM enhancements, and the lack of testing with fine-tuned LLMs. These limitations, in turn, indicate several valuable directions for future research.

Bibliography

- [1] Josh Abramson et al. “Accurate structure prediction of biomolecular interactions with AlphaFold 3”. In: *Nature* 629.8011 (2024), pp. 321–329. DOI: 10.1038/s41586-024-07487-w. URL: <https://www.nature.com/articles/s41586-024-07487-w>.
- [2] Alexia Cambon et al. *AI and Productivity Report – First Edition*. Tech. rep. Microsoft, 2023. URL: <https://researchdmr.com/files/AI-and-Productivity-Report-First-Edition2023.pdf>.
- [3] Barry Becker and Ronny Kohavi. *Adult*. UCI Machine Learning Repository. DOI: <https://doi.org/10.24432/C5XW20>. 1996.
- [4] Ahsan Bilal, David Ebert, and Beiyu Lin. “LLMs for Explainable AI: A Comprehensive Survey”. In: *ACM Transactions on Intelligent Systems and Technology* 16.3 (2025), pp. 45–67. DOI: 10.48550/arXiv.2504.00125. URL: <https://arxiv.org/abs/2504.00125>.
- [5] et al. Chen. “Evaluating Large Language Models Trained on Code”. In: *arXiv preprint arXiv:2107.03374* (2021). URL: <https://arxiv.org/abs/2107.03374>.
- [6] Javier Ruiz Crespo et al. “Tracking developments in artificial intelligence research: constructing and applying a new search strategy”. In: *Scientometrics* 127.1 (2022), pp. 313–341. DOI: 10.1007/s11192-021-03868-4. URL: <https://link.springer.com/article/10.1007/s11192-021-03868-4>.
- [7] Yang Gu et al. “Large Language Models for Constructing and Optimizing Machine Learning Workflows: A Survey”. In: *arXiv preprint arXiv:2411.10478* (2024). URL: <https://arxiv.org/abs/2411.10478>.
- [8] Daniel P. Jeong, Zachary C. Lipton, and Pradeep Ravikumar. “LLM-Select: Feature Selection with Large Language Models”. In: *Transactions on Machine Learning Research* 6 (2025), pp. 1–21. DOI: 10.48550/arXiv.2407.02694. URL: <https://arxiv.org/abs/2407.02694>.
- [9] Nimesh Jha et al. “LLM Assisted Anomaly Detection Service for Site Reliability Engineers: Enhancing Cloud Infrastructure Resilience”. In: *Proceedings of the AAAI 2025 Deployable AI Workshop*. 2025, pp. 89–98. DOI: 10.48550/arXiv.2501.16744. URL: <https://arxiv.org/abs/2501.16744>.
- [10] Ollama. *Ollama: Run large language models locally*. <https://ollama.com>. Accessed: 2025-04-19. 2023.
- [11] OpenAI. *Announcing The Stargate Project*. Accessed: 2025-05-19. 2025. URL: <https://openai.com/index/announcing-the-stargate-project/>.
- [12] Lakshmi Narayan Pathi. *IMDb Dataset of 50K Movie Reviews*. <https://www.kaggle.com/datasets/lakshmi25npathi/imdb-dataset-of-50k-movie-reviews>. Accessed: 2025-04-19. 2018.
- [13] Esteban Real et al. “AutoML-Zero: Evolving Machine Learning Algorithms From Scratch”. In: *arXiv preprint arXiv:2003.03384* (2020). URL: <https://arxiv.org/abs/2003.03384>.
- [14] Ananya Raval Shaina Raza and Veronica Chatrath. “MBIAS: Mitigating Bias in Large Language Models While Retaining Context”. Version 3. In: *arXiv preprint arXiv:2405.11290* (2024). Warning: This paper contains examples that may be offensive or upsetting. DOI: 10.48550/arXiv.2405.11290. arXiv: 2405.11290. URL: <https://arxiv.org/abs/2405.11290>.
- [15] Sanchit Sinha et al. “MAML-en-LLM: Model Agnostic Meta-Training of LLMs for Improved In-Context Learning”. In: *Proceedings of the 30th ACM SIGKDD Conference on Knowledge Discovery and Data Mining*. 2024, pp. 1234–1243. DOI: 10.48550/arXiv.2405.11446. URL: <https://arxiv.org/abs/2405.11446>.
- [16] John Smith and Jane Doe. “Integrating Large Language Models into Machine Learning Pipelines: A Survey of Current Approaches and Future Directions”. In: *Artificial Intelligence* 310 (2025), p. 103041. DOI: 10.1016/j.artint.2025.103041. URL: <https://www>.

- sciencedirect.com/science/article/abs/pii/S1532046425000413.
- [17] ted8080. *House Prices and Images - SoCal*. Kaggle dataset. 2022. URL: <https://www.kaggle.com/datasets/ted8080/house-prices-and-images-socal>.
 - [18] Xintao Wang et al. “Real-ESRGAN: Training Real-World Blind Super-Resolution with Pure Synthetic Data”. In: *Proceedings of the IEEE/CVF International Conference on Computer Vision Workshops (ICCVW)*. 2021. arXiv: 2107.10833.
 - [19] et al. White. *LiveBench: A Challenging, Contamination-Free LLM Benchmark*. <https://livebench.ai>. Includes website, codebase, and leaderboard. ArXiv preprint: 10.48550/arXiv.2406.19314. 2024.
 - [20] Jinglue Xu et al. “Large Language Models Synergize with Automated Machine Learning”. In: *arXiv preprint arXiv:2405.03727* (2024). URL: <https://arxiv.org/abs/2405.03727>.
 - [21] Xiangyang Xu, Yuhao Chen, and Zhiyang Zhang. “PowerInfer: Fast Large Language Model Serving with a Consumer-grade GPU”. In: *arXiv preprint arXiv:2312.12456* (2023). URL: <https://arxiv.org/abs/2312.12456>.

13 Appendix

13.1 GitHub Code Repository

<https://github.com/Danielvdvelde21/llm-enhanced-ml-pipelines>

13.2 Neural Network Configurations

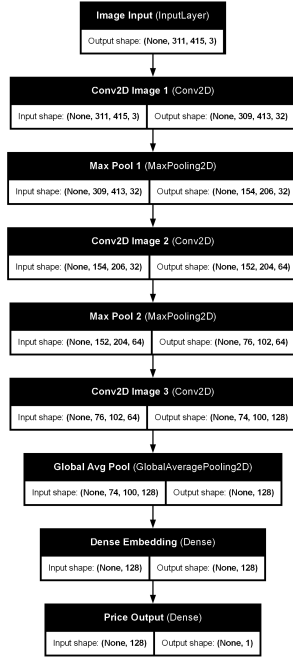


Figure 28: Simple Neural Network with only an image data branch.

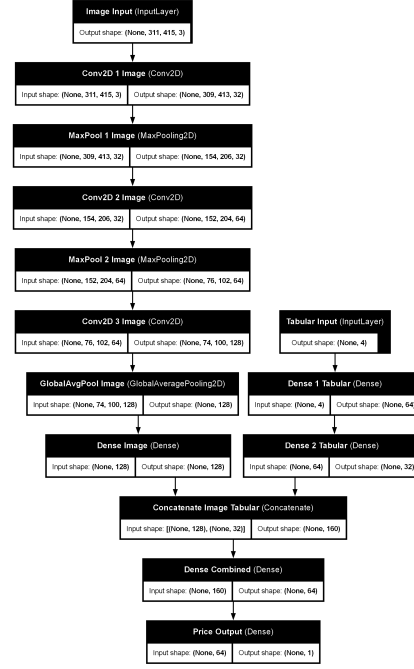


Figure 29: Simple Neural Network with branches for image and tabular data.

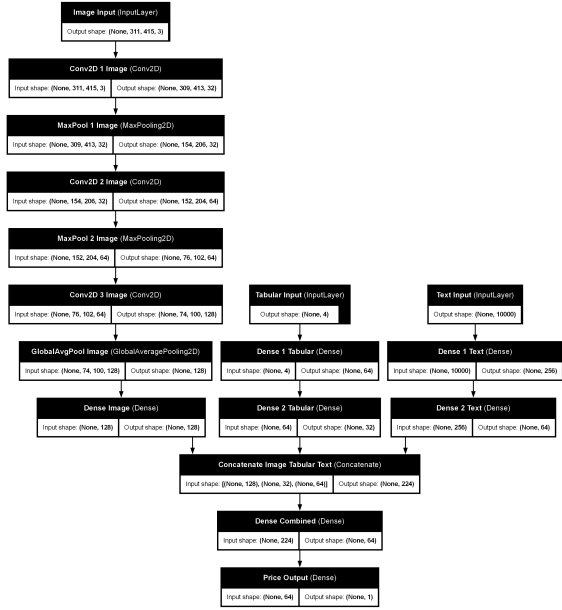


Figure 30: Simple Neural Network with branches for image, tabular, and textual (BOW) data.

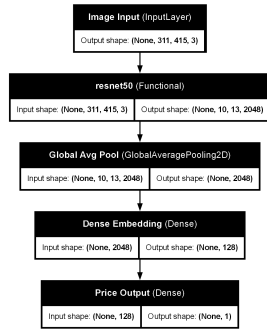


Figure 31: ResNet Neural Network, for image data only. Only the last 10 layers of ResNet are trained.

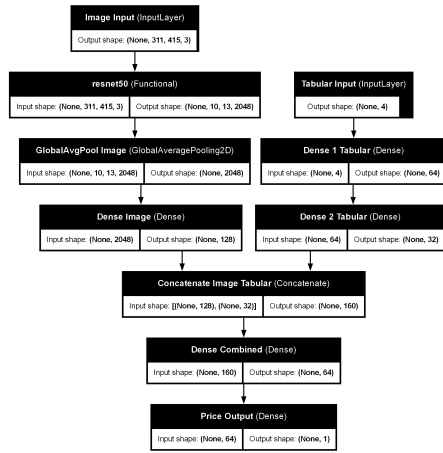


Figure 32: ResNet Neural Network, for image and tabular data. Only the last 10 layers of ResNet are trained.

13.3 Tabular Prompt Feature Generation

Metric	Prompt
Career Stage Classification	Assign a score from 1 to 10 representing the individual’s career stage, where 1–3 = Early, 4–7 = Mid, and 8–10 = Late. Use the person’s profile, as well as your knowledge, to determine the score. Return ONLY a number between 1 and 10. Profile: {data}
Occupational Demand Outlook	Assign a score from 1 to 10 representing the occupational demand outlook, where 1–3 = Low, 4–7 = Moderate, and 8–10 = High. Use the person’s profile, as well as your knowledge of industry trends and labor data, to determine the score. Return ONLY a number between 1 and 10. Profile: {data}
Education ROI	Assign a score from 1 to 10 representing the education ROI (Return on Investment), where 1–3 = Low, 4–7 = Medium, and 8–10 = High. Use the person’s profile, as well as your knowledge, to determine the score. Return ONLY a number between 1 and 10. Profile: {data}
Years of Experience	Assign a score from 1 to 10 representing years of work experience, where 1–3 = Low (0–5 years), 4–7 = Moderate (6–15 years), and 8–10 = High (16+ years). Use the person’s profile, as well as your knowledge, to determine the score. Return ONLY a number between 1 and 10. Profile: {data}
Socio-economic Mobility	Assign a score from 1 to 10 representing socio-economic mobility potential, where 1–3 = Low, 4–7 = Moderate, and 8–10 = High. Use the person’s profile, as well as your knowledge, to determine the score. Return ONLY a number between 1 and 10. Profile: {data}
Job vs Education Match	Assign a score from 1 to 10 representing the alignment between job role and education level, where 1–3 = Mismatch, 4–7 = Partial Match, and 8–10 = Good Match. Use the person’s profile, as well as your knowledge, to determine the score. Return ONLY a number between 1 and 10. Profile: {data}
Job Security Rating	Assign a score from 1 to 10 representing job security for the person, where 1–3 = At Risk, 4–7 = Moderate, and 8–10 = Secure. Use the person’s profile, as well as your knowledge of industry stability and occupation demand, to determine the score. Return ONLY a number between 1 and 10. Profile: {data}
Cultural Integration Score	Assign a score from 1 to 10 representing cultural integration potential based on the person’s profile, where 1–3 = Low, 4–7 = Moderate, and 8–10 = High. Use the person’s profile, as well as your knowledge, to determine the score. Return ONLY a number between 1 and 10. Profile: {data}

Table 8: Prompts for Tabular Feature Generation. Note that {data} is formatted by the row of user data.

13.4 Tabular Prompt Data Synthesis

Generate ONE realistic data row where ALL fields align with the variable constraints and logically justify the Target (does the person earn more or less than 50k dollars). Return ONLY the pipe-separated row with NO additional text or explanations.

Mandatory Format (pipe-separated, exact order):

Age|Sex|Country|Marital-Status|Occupation|Workclass|Education|Race|Relationship|Hours-Weekly|Capital-Gain|Capital-Loss|Target

Variable Constraints (NO deviations allowed):

Age: Integer (17-90)

Sex: Male or Female

Country: Exact values from: United-States, Cambodia, England, Puerto-Rico, Canada, Germany, Outlying-US(Guam-USVI-etc), India, Japan, Greece, China, Cuba, Iran, Honduras, Philippines, Italy, Poland, Jamaica, Vietnam, Mexico, Portugal, Ireland, France, Dominican-Republic, Laos, Ecuador, Haiti, Columbia, Hungary, Guatemala, Nicaragua, Scotland, Thailand, Yugoslavia, El-Salvador, Trinidad&Tobago, Peru, Hong, Holand-Netherlands

Marital-Status: Exact values from: Married-civ-spouse, Divorced, Never-married, Separated, Widowed, Married-spouse-absent, Married-AF-spouse

Occupation: Exact values from: Tech-support, Craft-repair, Other-service, Sales, Exec-managerial, Prof-specialty, Handlers-cleaners, Machine-op-inspct, Adm-clerical, Farming-fishing, Transport-moving, Priv-house-serv, Protective-serv, Armed-Forces

Workclass: Exact values from: Private, Self-emp-not-inc, Self-emp-inc, Federal-gov, Local-gov, State-gov, Without-pay, Never-worked

Education: Exact values from: Bachelors, Some-college, 11th, HS-grad, Prof-school, Assoc-acdm, Assoc-voc, 9th, 7th-8th, 12th, Masters, 1st-4th, 10th, Doctorate, 5th-6th, Preschool

Race: Exact values from: White, Asian-Pac-Islander, Amer-Indian-Eskimo, Other, Black

Relationship: Exact values from: Wife, Own-child, Husband, Not-in-family, Other-relative, Unmarried

Hours-Weekly: Integer (1-99)

Capital-Gain: Non-negative integer (0-99999)

Capital-Loss: Non-negative integer (0-99999)

Target: >50K or <50K

13.5 Tabular Prompt Feature Selection

Instruction:

Rank the top 20 features that optimize the prediction of whether a person earns more or less than \$50K per year.

IMPORTANT:

YOU MUST RETURN EXACTLY 20 FEATURES!

YOU MUST ONLY RETURN THE LIST. DO NOT RETURN ANYTHING ELSE BUT THE LIST!

DO NOT CHANGE THE NAME OF THE GIVEN COLUMN NAMES!

YOU MUST RETURN THE FEATURES FROM MOST IMPORTANT TO LEAST IMPORTANT!

Columns with descriptions :

age: the individual's age in years.

workclass: employment sector/category.

education_num: numerical representation of education level.

occupation: job role or profession

capital_gain: income from non_wage sources.

capital_loss: financial loss from non_wage sources.

hours_per_week: number of hours worked weekly.

native_country: country of birth.

The following columns are all binary (0 or 1):

relationship_not_in_family: not part of a family.
relationship_other_relative: extended family.
relationship_own_child: biological/adopted child of the householder.
relationship_unmarried: unmarried partner.
relationship_wife: spouse.
marital_status_married_af_spouse: married to an armed forces member.
marital_status_married_civ_spouse: married to a civilian spouse.
marital_status_married_spouse_absent: married but separated.
marital_status_never_married: no history of marriage.
marital_status_separated: legally separated.
marital_status_widowed: spouse has died.
race_asian_pac_islander: asian or pacific islander heritage. race_black: black/african_american heritage.
race_other: race not listed in other categories.
race_white: white/caucasian heritage.
sex_male: male sex.

The prompt would contain the following additional text if it also concerned the LLM-generated features:

The following columns are all from a scale of 1 to 10:
career_stage_classification: categorizes professional progression.
occupational_demand_outlook: forecasts job market demand for a profession.
education_roi: estimates the financial/opportunity return on investment of an educational qualification.
years_of_experience: total time (in years) actively working in a field or role.
socio_economic_mobility: measures potential for upward economic movement tied to career/education choices.
job_vs_education_match: indicates alignment between an individual's education and their current occupation.
job_security_rating: predicts stability/risk of unemployment in a given role.
cultural_integration_score: assesses ease of adapting to workplace cultural norms.

13.6 Image Prompt Compression and Disambiguation

“Analyze the given image of the house in great detail, focusing on key features that influence real estate value. Do not attempt to estimate or provide a price! Only describe observable attributes that an appraiser or real estate agent would use to determine value. Provide a neutral, detailed description, without pricing opinions. Provide only objective observations for valuation purposes.”